ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: ĐỒ ÁN CƠ SỞ CNTT

HỆ THỐNG ĐẶT LỊCH GIẶT LÀ TRỰC TUYẾN

Giảng viên hướng dẫn: TS. Nguyễn Lệ Thu

Sinh viên thực hiện: Ngô Xuân Tuấn – MSV: 23010019

Khoá: K17 – 2023 – 2027

Lớp tín chỉ: Đồ án cơ sở Công nghệ Thông tin -1-1-26(N01).TH

Chương trình đào tạo: Hệ chính quy

Hà Nội, tháng 10 năm 2026

LỜI CẢM ƠN

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc tới **TS. Nguyễn Lệ Thu** đã tận tình hướng dẫn, định hướng và góp ý cho tôi trong suốt quá trình thực hiện đề tài "Hệ thống đặt lịch giặt là trực tuyến".

Tôi xin chân thành cảm ơn quý thầy cô Trường Công nghệ Thông tin Phenikaa, Đại học Phenikaa đã truyền đạt những kiến thức nền tảng về lập trình, cơ sở dữ liệu và công nghệ phần mềm, là cơ sở để tôi hoàn thành đồ án này.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ tôi trong quá trình học tập. Do thời gian và kiến thức còn hạn chế, báo cáo khó tránh khỏi thiếu sót; tôi rất mong nhận được sự góp ý của quý thầy cô.

LỜI CAM ĐOAN

Tôi cam đoan **Đồ án cơ sở - Hệ thống Đặt lịch giặt là trực tuyến** là sản phẩm trí tuệ của riêng tôi. Mọi thông tin, dữ liệu, hình ảnh, etc. được sử dụng từ các nguồn khác đều được trích dẫn đầy đủ và có thể tìm thấy các tài liệu liên quan thông qua mục tài liệu tham khảo.

Tôi xin chịu trách nhiệm hoàn toàn về nội dung của **Đồ án cơ sở** mà tôi đã nộp.

Hà Nội, ngày … tháng … năm

Ngô Xuân Tuấn (ký tên)

PHÂN CÔNG NHIỆM VỤ ĐỒ ÁN

Danh sách các công việc/nhiệm vụ	Mô tả tóm tắt công việc
Công việc 1	Khảo sát, thu thập yêu cầu nghiệp vụ đặt lịch giặt là
Công việc 2	Phân tích, thiết kế hệ thống, cơ sở dữ liệu và API
Công việc 3	Cài đặt backend: xác thực, người dùng, địa chỉ, cửa hàng, lịch mở cửa, dịch vụ
Công việc 4	Thiết kế và cài đặt module đặt lịch
Công việc 5	Kiểm thử, hoàn thiện mã nguồn và viết báo cáo

TT	Mã sinh viên	Họ và tên	Nhiệm vụ phân công	Nội dung đã thực hiện	Đánh giá
1	23010019	Ngô Xuân Tuấn	Công việc 1	Tài liệu yêu cầu, quy trình	100%
			Công việc 2	ERD, kiến trúc, thiết kế API	100%
			Công việc 3	6 module, 40 endpoint	100%
			Công việc 4	Thiết kế xong; đang cài đặt	Đang thực hiện
			Công việc 5	Báo cáo, kịch bản kiểm thử	Đang thực hiện

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI 1
1.1. Lý do chọn đề tài 1
1.2. Mục tiêu của đề tài 1
1.3. Đối tượng, phạm vi và giới hạn của đề tài 2
1.3.1. Đối tượng sử dụng 2
1.3.2. Phạm vi thực hiện 2
1.3.3. Giới hạn 3
1.4. Phương pháp nghiên cứu và thực hiện 3
1.5. Bố cục báo cáo 3
1.6. Kết luận chương 1 4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 5
2.1. Kiến trúc client–server và RESTful API 5
2.2. Node.js, TypeScript và NestJS 5
2.3. PostgreSQL và Prisma ORM 6
2.4. Xác thực và phân quyền 6
2.4.1. Băm mật khẩu bằng bcrypt 6
2.4.2. JSON Web Token và cặp access token – refresh token 6
2.4.3. Băm token bằng HMAC-SHA256 7
2.4.4. Phân quyền theo vai trò (RBAC) 7
2.5. Redis và giới hạn tần suất (rate limiting) 7
2.6. Dịch vụ bản đồ và tính khoảng cách 7
2.7. Kiểm tra dữ liệu đầu vào 8
2.8. Công cụ phát triển 8
2.9. Kết luận chương 2 8
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 9
3.1. Mô tả bài toán và quy trình nghiệp vụ 9
3.2. Các tác nhân 9
3.3. Yêu cầu chức năng 10
3.4. Yêu cầu phi chức năng 11
3.5. Biểu đồ use case 11
3.6. Kiến trúc hệ thống 12
3.7. Thiết kế cơ sở dữ liệu 14

3.7.1. Mô hình thực thể – quan hệ 14
3.7.2. Danh sách bảng 16
3.7.3. Một số quyết định thiết kế 16
3.8. Thiết kế API 17
3.9. Thiết kế các luồng nghiệp vụ chính 18
3.9.1. Xác thực và quản lý phiên 18
3.9.2. Sinh khung giờ còn trống 19
3.9.3. Quy trình đặt lịch 20
3.9.4. Vòng đời trạng thái của đơn 22
3.10. Thiết kế bảo mật 23
3.11. Kết luận chương 3 24
CHƯƠNG 4: TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG 25
4.1. Giới thiệu chương 25
4.2. Môi trường phát triển 25
4.3. Cấu trúc mã nguồn 25
4.4. Cài đặt các module 25
4.4.1. Khởi tạo ứng dụng và cấu hình 25
4.4.2. Module xác thực và người dùng 26
4.4.3. Module địa chỉ 26
4.4.4. Module cửa hàng 27
4.4.5. Module lịch mở cửa và khung giờ 27
4.4.6. Module dịch vụ giặt 27
4.4.7. Chuẩn bị cho module đặt lịch 28
4.5. Tiến độ triển khai 28
4.6. Kiểm thử 29
4.6.1. Chiến lược kiểm thử 29
4.6.2. Kết quả kiểm tra tự động 29
4.6.3. Kịch bản kiểm thử chức năng API 29
4.7. Đánh giá kết quả đạt được 32
4.8. Hạn chế của hệ thống 32
4.9. Kết luận chương 4 32
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 33
5.1. Kết luận 33

5.2. Hạn chế của đề tài 33
5.3. Hướng phát triển 33
5.4. Kết lời 34
TÀI LIỆU THAM KHẢO 35
PHỤ LỤC 36
PHỤ LỤC A. DANH SÁCH API 36
PHỤ LỤC B. TỪ ĐIỂN DỮ LIỆU 37
PHỤ LỤC C. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY 46
PHỤ LỤC D. BIẾN MÔI TRƯỜNG 47
PHỤ LỤC E. VÍ DỤ YÊU CẦU API 47

DANH MỤC TỪ VIẾT TẮT

Chữ viết tắt	Giải thích
API	Application Programming Interface – Giao diện lập trình ứng dụng
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete – Tạo, đọc, sửa, xóa
CSDL	Cơ sở dữ liệu
DI	Dependency Injection – Tiêm phụ thuộc
DTO	Data Transfer Object – Đối tượng truyền dữ liệu
ERD	Entity Relationship Diagram – Sơ đồ thực thể – quan hệ
GVHD	Giảng viên hướng dẫn
HMAC	Hash-based Message Authentication Code
HTTP(S)	HyperText Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
NV	Nhân viên giặt là (vai trò laundry_staff)
ORM	Object–Relational Mapping – Ánh xạ đối tượng – quan hệ
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
RBAC	Role-Based Access Control – Phân quyền theo vai trò
REST	Representational State Transfer
SQL	Structured Query Language
TTL	Time To Live – Thời gian sống
UQ	Unique – Ràng buộc duy nhất
UTC	Coordinated Universal Time – Giờ phối hợp quốc tế

DANH MỤC BẢNG BIỂU, HÌNH VẼ

Bảng 1.1. Phạm vi chức năng và mức độ hoàn thành 2
Bảng 2.1. Công nghệ và công cụ sử dụng 8
Bảng 3.1. Các tác nhân của hệ thống 9
Bảng 3.2. Danh sách yêu cầu chức năng 10
Bảng 3.3. Yêu cầu phi chức năng 11
Hình 3.1. Sơ đồ use case tổng quát của hệ thống 12
Hình 3.2. Kiến trúc tổng quát của hệ thống 13
Bảng 3.4. Các module và trách nhiệm 13
Hình 3.3. Sơ đồ thực thể – quan hệ (rút gọn thuộc tính) 15
Bảng 3.5. Các bảng dữ liệu và trạng thái sử dụng 16
Bảng 3.6. Tổng hợp nhóm API 17
Hình 3.4. Sơ đồ trình tự đăng nhập, làm mới phiên và đăng xuất 18
Bảng 3.7. Chính sách giới hạn tần suất 19
Bảng 3.8. Quy tắc ghi đè của ngày ngoại lệ 19
Hình 3.5. Lưu đồ tạo đơn đặt lịch 21
Bảng 3.9. Cách tính giá theo loại dịch vụ 21
Hình 3.6. Sơ đồ trạng thái của đơn đặt lịch 23
Bảng 3.10. Các mối đe dọa và biện pháp bảo mật 23
Bảng 4.1. Môi trường phát triển 25
Bảng 4.2. Tiến độ triển khai theo module 28
Bảng 4.3. Kết quả kiểm tra tự động 29
Bảng 4.4. Kịch bản kiểm thử module Auth 29
Bảng 4.5. Kịch bản kiểm thử module User và Address 30
Bảng 4.6. Kịch bản kiểm thử module Store 30
Bảng 4.7. Kịch bản kiểm thử module lịch mở cửa và dịch vụ 31
Bảng 4.8. Đối chiếu kết quả với mục tiêu 32

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Giặt là là nhu cầu thường xuyên của sinh viên, người đi làm và các hộ gia đình ở đô thị, những người ít có thời gian tự giặt giũ. Tại nhiều cửa hàng giặt là quy mô nhỏ, việc tiếp nhận đơn vẫn diễn ra thủ công qua điện thoại, tin nhắn hoặc sổ ghi chép. Cách làm này bộc lộ nhiều bất cập:

- Khách hàng không biết cửa hàng còn nhận đồ vào khung giờ nào, phải gọi hỏi nhiều lần và dễ phải chờ đợi.
- Cửa hàng khó kiểm soát công suất: giờ cao điểm bị quá tải, giờ thấp điểm lại thưa khách.
- Báo giá không thống nhất, dễ phát sinh tranh chấp khi giá thay đổi giữa lúc đặt và lúc nhận đồ.
- Khó theo dõi tiến trình xử lý từng đơn, dễ nhầm lẫn hoặc thất lạc đồ; không có dữ liệu để thống kê, đối soát thanh toán.

Việc số hóa quy trình đặt lịch giặt là bằng một hệ thống trực tuyến cho phép khách hàng tự chọn cửa hàng, dịch vụ và khung giờ phù hợp; cho phép cửa hàng quản lý lịch mở cửa, số lượng đơn nhận tối đa theo từng khung giờ và minh bạch bảng giá. Toàn bộ dữ liệu được lưu tập trung, thuận lợi cho việc theo dõi, thống kê và mở rộng.

Về mặt chuyên môn, đây là đề tài có đủ độ phức tạp để vận dụng các kiến thức đã học: phân tích thiết kế hệ thống, thiết kế cơ sở dữ liệu quan hệ, xây dựng API theo chuẩn REST, xác thực và phân quyền, xử lý đồng thời khi nhiều người cùng giữ một khung giờ, và tích hợp dịch vụ bên thứ ba (bản đồ, thanh toán). Vì vậy, tôi chọn đề tài **"Hệ thống đặt lịch giặt là trực tuyến"** làm đồ án.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát: xây dựng phần lõi (backend) của hệ thống đặt lịch giặt là trực tuyến, cung cấp các API an toàn, nhất quán dữ liệu cho các ứng dụng khách (web, di động) và trang quản trị.

Mục tiêu cụ thể:

- Phân tích nghiệp vụ, xác định các tác nhân (khách hàng, nhân viên giặt là, shipper, quản trị viên) và quy trình đặt lịch giặt là.
- Thiết kế cơ sở dữ liệu quan hệ (23 bảng) và kiến trúc backend chia theo module.

- Xây dựng API cho các chức năng: xác thực và phân quyền; quản lý người dùng và sổ địa chỉ; quản lý cửa hàng; lịch mở cửa, ngày ngoại lệ và khung giờ còn trống; danh mục dịch vụ và bảng giá.
- Thiết kế quy trình đặt lịch bảo đảm không nhận quá công suất của khung giờ và giá được chốt tại thời điểm đặt.
- Bảo mật dữ liệu người dùng: băm mật khẩu và token, giới hạn tần suất gọi API, kiểm tra chặt dữ liệu đầu vào.
- Kiểm thử, rà soát mã nguồn và hoàn thiện tài liệu.

1.3. Đối tượng, phạm vi và giới hạn của đề tài

1.3.1. Đối tượng sử dụng

Hệ thống phục vụ bốn nhóm người dùng, tương ứng bốn vai trò trong cơ sở dữ liệu: **khách hàng** (đặt lịch giặt), **nhân viên giặt là** (quản lý cửa hàng được phân công), **shipper** (lấy và giao đồ) và **quản trị viên** (quản lý toàn hệ thống).

1.3.2. Phạm vi thực hiện

Đề tài tập trung vào phần backend (REST API) xây dựng bằng NestJS, TypeScript, PostgreSQL và Redis. Mức độ hoàn thành từng nhóm chức năng được tổng hợp trong Bảng 1.1. Hệ thống được thiết kế đầy đủ cho toàn bộ quy trình nghiệp vụ, nhưng việc triển khai được thực hiện theo từng giai đoạn.

Bảng 1.1. Phạm vi chức năng và mức độ hoàn thành

Nhóm chức năng	Nội dung chính	Trạng thái
Xác thực & người dùng	Đăng ký, xác minh email, đăng nhập, làm mới phiên, đăng xuất, quên/đặt lại mật khẩu, hồ sơ cá nhân	Đã triển khai
Địa chỉ	Sổ địa chỉ, địa chỉ mặc định, tự động xác định tọa độ	Đã triển khai
Cửa hàng	CRUD cửa hàng, tìm kiếm, sắp xếp theo khoảng cách, gán nhân viên	Đã triển khai
Lịch mở cửa & slot	Lịch tuần, ngày ngoại lệ, sinh khung giờ còn trống	Đã triển khai
Dịch vụ & bảng giá	Dịch vụ riêng của cửa hàng và dịch vụ dùng chung	Đã triển khai
Đặt lịch (Booking)	Tạo, xem, hủy đặt lịch; chốt giá; giữ chỗ chống vượt công suất	Đã thiết kế, đang triển khai
Thanh toán, khuyến mãi	VNPay/MoMo/Stripe/tiền mặt; mã giảm giá	Định hướng
Giao nhận, thông báo, đánh giá, khiếu nại	Chuyến lấy/giao đồ, thông báo, đánh giá, khiếu nại, danh sách chờ	Định hướng

1.3.3. Giới hạn

- Chưa có giao diện người dùng; việc thử nghiệm thực hiện qua công cụ gọi API (Postman).
- Chưa tích hợp cổng thanh toán thật và chưa gửi email thật; ở môi trường phát triển, mã xác minh được ghi ra nhật ký máy chủ.
- Module đặt lịch mới ở mức thiết kế chi tiết và chuẩn bị cơ sở dữ liệu (đã bổ sung các cột lưu giá tại thời điểm đặt).
- Khoảng cách giữa khách và cửa hàng tính theo đường chim bay (công thức Haversine), chưa tính đường đi thực tế.

1.4. Phương pháp nghiên cứu và thực hiện

- **Tìm hiểu nghiệp vụ:** tham khảo cách vận hành thông thường của các tiệm giặt là và dịch vụ giặt là trực tuyến để xác định quy trình, các tác nhân và quy tắc nghiệp vụ (chính sách hủy, hoàn tiền, khung giờ nhận đồ).
- **Nghiên cứu công nghệ:** tìm hiểu NestJS, Prisma, PostgreSQL, Redis, JWT và các dịch vụ bản đồ mở (Nominatim, OSRM) từ tài liệu chính thức.
- **Phân tích và thiết kế:** mô hình hóa bằng sơ đồ use case, sơ đồ thực thể – quan hệ, sơ đồ trình tự và lưu đồ.
- **Phát triển lặp theo module:** với mỗi module thực hiện chu trình thiết kế → cài đặt → rà soát mã (lỗi bảo mật, xử lý đồng thời) → kiểm thử.
- **Kiểm thử:** kiểm tra tĩnh (TypeScript chế độ nghiêm ngặt, oxlint), kiểm thử đơn vị cho các hàm thuần (Vitest) và kiểm thử API theo kịch bản.
- **Công cụ hỗ trợ:**
  - Hỗ trợ code/co-work: Claude, ChatGPT,…
  - Hỗ trợ vẽ diagram: Draw.io, Visual Paradigm,…
  - Hỗ trợ UI/UX: Figma, Ant Design,…

1.5. Bố cục báo cáo

Ngoài phần mở đầu, danh mục, tài liệu tham khảo và phụ lục, báo cáo gồm 5 chương:

- **Chương 1 – Tổng quan đề tài:** lý do chọn đề tài, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2 – Cơ sở lý thuyết và công nghệ sử dụng:** các kiến thức và công cụ nền tảng của hệ thống.
- **Chương 3 – Phân tích và thiết kế hệ thống:** yêu cầu, use case, kiến trúc, cơ sở dữ liệu, API và các luồng nghiệp vụ chính.
- **Chương 4 – Triển khai, kiểm thử và đánh giá:** cài đặt các module, kết quả kiểm thử và đánh giá mức độ đạt mục tiêu.

- **Chương 5 – Kết luận và hướng phát triển.**

1.6. Kết luận chương 1

Chương này đã trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp thực hiện. Đề tài giải quyết bài toán số hóa quy trình đặt lịch giặt là, tập trung vào tính đúng đắn của dữ liệu (không vượt công suất, giá nhất quán) và an toàn thông tin người dùng. Chương tiếp theo trình bày cơ sở lý thuyết và các công nghệ được sử dụng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Kiến trúc client–server và RESTful API

Hệ thống được xây dựng theo mô hình client–server: các ứng dụng khách (web, di động, Postman) gửi yêu cầu HTTP tới máy chủ, máy chủ xử lý nghiệp vụ, truy xuất cơ sở dữ liệu và trả về dữ liệu dạng JSON. Giao diện lập trình tuân theo phong cách REST (Representational State Transfer) do Roy Fielding đề xuất: mỗi tài nguyên có một địa chỉ riêng, thao tác được thể hiện qua phương thức HTTP (GET để đọc, POST để tạo, PUT/PATCH để cập nhật, DELETE để xóa) và máy chủ không lưu trạng thái phiên giữa các yêu cầu (stateless).

Các quy ước API được áp dụng thống nhất trong đề tài:

- Mọi đường dẫn có tiền tố `/api`; tài nguyên lồng nhau thể hiện quan hệ, ví dụ `/stores/:storeId/services`.
- Mã trạng thái HTTP: `200/201` thành công; `400` dữ liệu không hợp lệ; `401` chưa xác thực; `403` không đủ quyền; `404` không tồn tại; `409` xung đột (trùng dữ liệu, hết chỗ); `429` gọi quá nhiều lần.
- Danh sách có phân trang trả về `{ data, meta: { total, page, limit, totalPages } }`.

2.2. Node.js, TypeScript và NestJS

Node.js là môi trường chạy JavaScript phía máy chủ theo mô hình không đồng bộ, hướng sự kiện, phù hợp với các ứng dụng nhiều thao tác vào/ra như API web. **TypeScript** bổ sung hệ thống kiểu tĩnh cho JavaScript; đề tài bật chế độ `strict` để phát hiện sớm lỗi kiểu dữ liệu ngay khi biên dịch.

NestJS là framework xây dựng ứng dụng Node.js có kiến trúc rõ ràng, lấy cảm hứng từ Angular. Các khái niệm chính được sử dụng:

- **Module:** đơn vị tổ chức mã nguồn; mỗi nghiệp vụ (Auth, Store, Booking, …) là một module đóng gói controller và service.
- **Controller:** tiếp nhận yêu cầu HTTP, gọi service và trả kết quả; không chứa logic nghiệp vụ phức tạp.
- **Provider/Service và Dependency Injection:** logic nghiệp vụ nằm trong service, được NestJS tự động inject vào nơi cần dùng, giúp tách lớp và dễ kiểm thử.
- **Guard:** quyết định yêu cầu có được xử lý hay không (xác thực JWT, kiểm tra vai trò, giới hạn tần suất).
- **Pipe:** kiểm tra và chuyển đổi dữ liệu đầu vào (ValidationPipe, ParseIntPipe).

- **ConfigModule:** nạp và kiểm tra biến môi trường tại thời điểm khởi động.

2.3. PostgreSQL và Prisma ORM

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ đầy đủ tính chất ACID, khóa ngoại, kiểu enum và khóa ở mức dòng. Tính chất này đặc biệt quan trọng với bài toán đặt lịch: khi nhiều khách cùng đặt một khung giờ, hệ thống cần khóa dữ liệu bằng `SELECT … FOR UPDATE` bên trong một transaction để không nhận vượt công suất.

Prisma là ORM thế hệ mới cho TypeScript. Lược đồ dữ liệu được khai báo trong tệp `schema.prisma`; Prisma sinh ra client có kiểu dữ liệu đầy đủ, giúp truy vấn an toàn kiểu. Các tính năng được dùng trong đề tài:

- `$transaction` để gom nhiều thao tác ghi thành một đơn vị nguyên tử (tất cả cùng thành công hoặc cùng hoàn tác).
- Kiểu `Decimal` cho tiền tệ và tọa độ, tránh sai số của số thực dấu phẩy động.
- `@updatedAt` tự động cập nhật thời điểm sửa đổi bản ghi.
- `prisma db push` đồng bộ lược đồ với cơ sở dữ liệu trong giai đoạn phát triển; adapter `@prisma/adapter-pg` kết nối trực tiếp tới PostgreSQL.

2.4. Xác thực và phân quyền

2.4.1. Băm mật khẩu bằng bcrypt

Mật khẩu không bao giờ được lưu dạng gốc. Hệ thống dùng **bcrypt**, một hàm băm được thiết kế chậm có chủ đích và tự sinh muối (salt) cho từng mật khẩu; độ chậm được điều chỉnh bằng tham số cost (`SALT_ROUNDS`, mặc định 10), làm tăng chi phí của tấn công dò mật khẩu hàng loạt. bcrypt chỉ xử lý tối đa 72 byte đầu vào nên độ dài mật khẩu được giới hạn 72 ký tự.

2.4.2. JSON Web Token và cặp access token – refresh token

JWT (RFC 7519) là chuỗi gồm ba phần `header.payload.signature`, được ký bằng thuật toán HMAC để máy chủ xác minh tính toàn vẹn mà không cần tra cứu cơ sở dữ liệu. Hệ thống dùng hai loại token:

- **Access token (JWT)** sống ngắn (mặc định 15 phút), gửi trong tiêu đề `Authorization: Bearer`, chứa mã người dùng, email và vai trò.
- **Refresh token** là chuỗi ngẫu nhiên 32 byte, sống 30 ngày, lưu trong cookie `HttpOnly` (JavaScript phía trình duyệt không đọc được), `SameSite=Lax`, `Secure` khi chạy production. Mỗi lần làm mới, refresh token cũ bị thu hồi và cấp token mới (**rotation**), giúp hạn chế hậu quả khi một token bị lộ.

2.4.3. Băm token bằng HMAC-SHA256

Các token dùng một lần (xác minh email, đặt lại mật khẩu) và refresh token không được lưu dạng gốc trong cơ sở dữ liệu mà lưu giá trị **HMAC-SHA256** (RFC 2104) với khóa bí mật riêng. Khi cơ sở dữ liệu bị lộ, kẻ tấn công chỉ có giá trị băm và không thể dùng trực tiếp để đặt lại mật khẩu hay mạo danh phiên đăng nhập.

2.4.4. Phân quyền theo vai trò (RBAC)

Quyền truy cập được xác định theo vai trò (`admin`, `laundry_staff`, `shipper`, `customer`). Decorator `@Roles(...)` khai báo vai trò được phép, `RoleGuard` đối chiếu với vai trò trong access token. Với nhân viên giặt là, ngoài vai trò còn kiểm tra thêm quan hệ phân công (bảng `staff_assignments`) để nhân viên chỉ thao tác được trên cửa hàng của mình.

2.5. Redis và giới hạn tần suất (rate limiting)

Redis là kho dữ liệu khóa–giá trị lưu trong bộ nhớ, tốc độ rất cao, hỗ trợ thời gian sống (TTL) cho khóa. Hệ thống dùng Redis để đếm số lần gọi các API nhạy cảm (đăng nhập, đăng ký, quên mật khẩu, …) theo cửa sổ cố định: mỗi lần gọi thực hiện `INCR` và đặt `EXPIRE` trong cùng một script Lua nên thao tác là nguyên tử, không bị lỗi khi nhiều yêu cầu đến đồng thời. Khóa đếm được tạo từ HMAC của địa chỉ IP hoặc email nên không lưu thông tin cá nhân dạng rõ. Vượt ngưỡng, máy chủ trả mã `429`.

2.6. Dịch vụ bản đồ và tính khoảng cách

Hệ thống dùng hai dịch vụ dựa trên dữ liệu OpenStreetMap, không yêu cầu khóa API:

- **Nominatim** – chuyển địa chỉ văn bản thành tọa độ (geocoding). Chính sách sử dụng yêu cầu khai báo `User-Agent` và không quá 1 yêu cầu mỗi giây; hệ thống đặt thời gian chờ 8 giây và xử lý lỗi theo hướng "cố gắng hết sức" (best-effort) để lỗi bản đồ không chặn việc lưu dữ liệu.
- **OSRM** – tính quãng đường và thời gian di chuyển giữa hai điểm (đã có hàm hỗ trợ, dự kiến dùng khi tính phí giao nhận).

Để sắp xếp cửa hàng theo khoảng cách, hệ thống dùng **công thức Haversine** tính khoảng cách đường chim bay giữa hai điểm trên mặt cầu:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)}\right), \quad \text{với } R \approx 6371 \text{ km},$$

φ là vĩ độ, λ là kinh độ (radian).

2.7. Kiểm tra dữ liệu đầu vào

Thư viện **class-validator** và **class-transformer** cho phép khai báo quy tắc kiểm tra bằng decorator ngay trên các lớp DTO (Data Transfer Object). `ValidationPipe` được bật toàn cục với `whitelist` (loại bỏ trường không khai báo) và `forbidNonWhitelisted` (từ chối yêu cầu có trường lạ). Cơ chế này ngăn kiểu tấn công gán đè trường (mass assignment), chẳng hạn gửi thêm `role: admin` khi đăng ký.

2.8. Công cụ phát triển

Các công cụ và phiên bản chính được tổng hợp trong Bảng 2.1.

Bảng 2.1. Công nghệ và công cụ sử dụng

Công nghệ / công cụ	Phiên bản	Vai trò
Node.js	24.x	Môi trường chạy JavaScript phía máy chủ
TypeScript	6.x (strict)	Ngôn ngữ lập trình
NestJS	12.x	Framework backend
Prisma ORM / adapter-pg	7.10	Truy cập cơ sở dữ liệu, transaction
PostgreSQL	—	Cơ sở dữ liệu quan hệ
Redis (ioredis)	6.x	Đếm giới hạn tần suất
bcrypt	6.x	Băm mật khẩu
passport-jwt, @nestjs/jwt	4.x / 12.x	Xác thực JWT
class-validator / class-transformer	0.15 / 0.5	Kiểm tra dữ liệu đầu vào
Vitest	4.x	Kiểm thử đơn vị
oxlint, Prettier	—	Kiểm tra và định dạng mã nguồn
pnpm, Git, Postman, VS Code	—	Quản lý gói, phiên bản, thử nghiệm API, soạn thảo

2.9. Kết luận chương 2

Chương 2 đã trình bày các cơ sở lý thuyết và công nghệ nền tảng: kiến trúc REST, NestJS, PostgreSQL/Prisma, cơ chế xác thực JWT kết hợp refresh token xoay vòng, băm mật khẩu và token, giới hạn tần suất bằng Redis, dịch vụ bản đồ và kiểm tra dữ liệu đầu vào. Đây là nền tảng để thiết kế hệ thống ở chương tiếp theo.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Mô tả bài toán và quy trình nghiệp vụ

Hệ thống kết nối khách hàng với các cửa hàng giặt là. Quy trình tổng thể của một đơn giặt gồm các bước sau:

- **Chuẩn bị:** khách đăng ký, xác minh email, đăng nhập và lưu một hoặc nhiều địa chỉ nhận/giao đồ.
- **Chọn cửa hàng và dịch vụ:** khách tìm cửa hàng theo tên hoặc theo khoảng cách gần nhất, xem bảng giá các dịch vụ.
- **Chọn khung giờ:** khách chọn ngày, hệ thống hiển thị các khung giờ còn trống dựa trên lịch mở cửa và số đơn đã nhận.
- **Đặt lịch:** khách chọn hình thức (tự mang đến cửa hàng hoặc lấy/giao tại nhà); hệ thống giữ chỗ, chốt giá và tạo đơn ở trạng thái chờ thanh toán.
- **Thanh toán và xác nhận:** sau khi thanh toán, đơn được xác nhận.
- **Xử lý:** lấy đồ → giặt → sẵn sàng giao → giao trả → hoàn tất. Đơn có thể bị hủy hoặc ghi nhận khách không đến.
- **Sau đơn:** khách đánh giá cửa hàng hoặc gửi khiếu nại nếu có sự cố.

Các quy tắc nghiệp vụ cốt lõi được rút ra từ quy trình trên:

- **R1.** Mỗi cửa hàng có lịch mở cửa theo thứ trong tuần; mỗi ngày chia thành các khung giờ (slot) có độ dài và sức chứa riêng.
- **R2.** Ngày ngoại lệ (nghỉ lễ, đổi giờ) ghi đè lịch tuần của ngày đó.
- **R3.** Mỗi đơn chiếm một chỗ trong khung giờ; đơn đã hủy hoặc khách không đến không chiếm chỗ.
- **R4.** Giá được chốt tại thời điểm đặt; thay đổi giá dịch vụ sau đó không ảnh hưởng đơn đã tạo.
- **R5.** Hủy đơn được hoàn tiền theo chính sách của từng cửa hàng (số giờ tối thiểu trước giờ lấy đồ và phần trăm hoàn).
- **R6.** Dịch vụ có thể thuộc riêng một cửa hàng hoặc dùng chung cho mọi cửa hàng.
- **R7.** Nhân viên chỉ được quản lý cửa hàng mà quản trị viên đã phân công.

3.2. Các tác nhân

Bảng 3.1. Các tác nhân của hệ thống

Tác nhân	Vai trò (DB)	Mô tả và quyền chính
Khách hàng	customer	Quản lý hồ sơ, sổ địa chỉ; tìm cửa hàng, xem dịch vụ và khung giờ; đặt, hủy, thanh toán và theo dõi đơn; đánh giá, khiếu nại.

Tác nhân	Vai trò (DB)	Mô tả và quyền chính
Nhân viên giặt là	laundry_staff	Quản lý thông tin, lịch mở cửa, ngày ngoại lệ và dịch vụ của cửa hàng được phân công; cập nhật trạng thái xử lý đơn.
Shipper	shipper	Nhận và cập nhật kết quả các chuyến lấy/giao đồ được giao.
Quản trị viên	admin	Quản lý người dùng; tạo/vô hiệu hóa cửa hàng; gán nhân viên; quản lý dịch vụ dùng chung, khuyến mãi; xem báo cáo.
Khách vãng lai	(chưa đăng nhập)	Xem danh sách cửa hàng, dịch vụ, lịch mở cửa và khung giờ còn trống.

3.3. Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo module và đánh dấu mức độ hoàn thành ở Bảng 3.2.

Bảng 3.2. Danh sách yêu cầu chức năng

Mã	Chức năng	Tác nhân	Trạng thái
FR01	Đăng ký tài khoản (kiểm tra email trùng, băm mật khẩu)	Khách	Đã triển khai
FR02	Xác minh email bằng token một lần (hiệu lực 15 phút)	Khách	Đã triển khai
FR03	Đăng nhập, cấp access token và refresh token	Mọi tài khoản	Đã triển khai
FR04	Làm mới phiên (rotation) và đăng xuất	Mọi tài khoản	Đã triển khai
FR05	Quên mật khẩu và đặt lại mật khẩu	Mọi tài khoản	Đã triển khai
FR06	Xem và cập nhật hồ sơ cá nhân	Mọi tài khoản	Đã triển khai
FR07	Xem danh sách và chi tiết người dùng	Admin	Đã triển khai
FR08	Quản lý sổ địa chỉ, địa chỉ mặc định, tự tính tọa độ	Khách	Đã triển khai
FR09	Tìm cửa hàng theo tên/địa chỉ, sắp xếp theo khoảng cách, xem chi tiết	Mọi người	Đã triển khai
FR10	Tạo, sửa, vô hiệu hóa cửa hàng	Admin; NV được gán (chỉ sửa)	Đã triển khai
FR11	Gán và gỡ nhân viên khỏi cửa hàng	Admin	Đã triển khai
FR12	Thiết lập lịch mở cửa theo tuần	Admin, NV được gán	Đã triển khai
FR13	Quản lý ngày ngoại lệ (nghỉ, đổi giờ)	Admin, NV được gán	Đã triển khai
FR14	Xem khung giờ còn trống theo ngày	Mọi người	Đã triển khai
FR15	Quản lý dịch vụ của cửa hàng (thêm, sửa, ngừng)	Admin, NV được gán	Đã triển khai
FR16	Xem danh sách và chi tiết dịch vụ, bảng giá	Mọi người	Đã triển khai
FR17	Xem và sửa dịch vụ dùng chung	Admin	Đã triển khai
FR18	Đặt lịch giặt (giữ chỗ, chốt giá, áp mã khuyến mãi)	Khách	Đang triển khai

Mã	Chức năng	Tác nhân	Trạng thái
FR19	Xem danh sách và chi tiết đơn của mình / của cửa hàng	Khách, NV, Admin	Đang triển khai
FR20	Hủy đơn và tính tỷ lệ hoàn tiền theo chính sách	Khách	Đang triển khai
FR21	Cập nhật trạng thái xử lý đơn	NV	Định hướng
FR22	Thanh toán trực tuyến (VNPay, MoMo, Stripe) và tiền mặt	Khách	Định hướng
FR23	Quản lý mã khuyến mãi	Admin	Định hướng
FR24	Quản lý chuyến lấy/giao đồ	Shipper, NV	Định hướng
FR25	Thông báo theo sự kiện (xác nhận, nhắc lịch, giao hàng, …)	Mọi người	Định hướng
FR26	Đánh giá cửa hàng sau đơn	Khách	Định hướng
FR27	Gửi và xử lý khiếu nại	Khách, Admin	Định hướng
FR28	Quản lý máy giặt, danh sách chờ khung giờ, nhật ký thao tác	NV, Admin	Định hướng

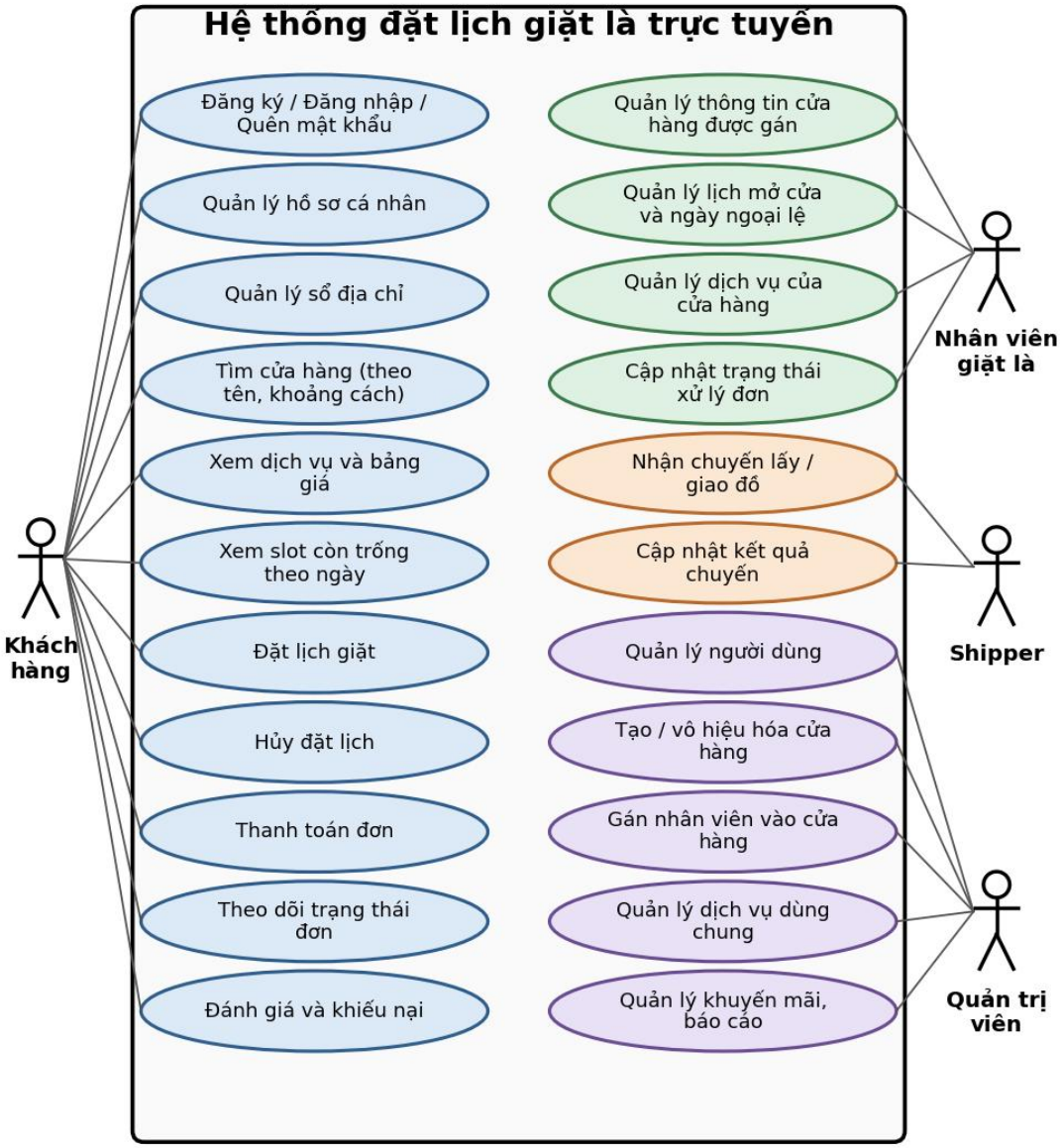
3.4. Yêu cầu phi chức năng

Bảng 3.3. Yêu cầu phi chức năng

Nhóm	Yêu cầu
Bảo mật	Mật khẩu và token lưu dạng băm; giới hạn tần suất các API nhạy cảm; phân quyền theo vai trò và theo phân công; không trả về dữ liệu nhạy cảm (password_hash).
Toàn vẹn dữ liệu	Các thao tác ghi nhiều bước dùng transaction; không nhận vượt sức chứa khung giờ; giá được chốt tại thời điểm đặt; ràng buộc khóa ngoại và duy nhất ở mức cơ sở dữ liệu.
Hiệu năng	API danh sách có phân trang (tối đa 100 bản ghi/trang); truy vấn có chỉ mục theo khóa ngoại và trạng thái; gọi dịch vụ ngoài có thời gian chờ 8 giây.
Độ tin cậy	Lỗi dịch vụ bản đồ không làm hỏng thao tác lưu (tọa độ để trống); mọi lỗi trả về mã HTTP và thông điệp rõ ràng.
Khả năng bảo trì	Kiến trúc module; DTO kiểm tra dữ liệu đầu vào; hàm thuần tách riêng để kiểm thử; mã nguồn kiểm tra bằng TypeScript strict và oxlint.
Khả năng cấu hình	Toàn bộ bí mật và tham số lấy từ biến môi trường, kiểm tra đầy đủ khi khởi động; không có giá trị bí mật mặc định trong mã.

3.5. Biểu đồ use case

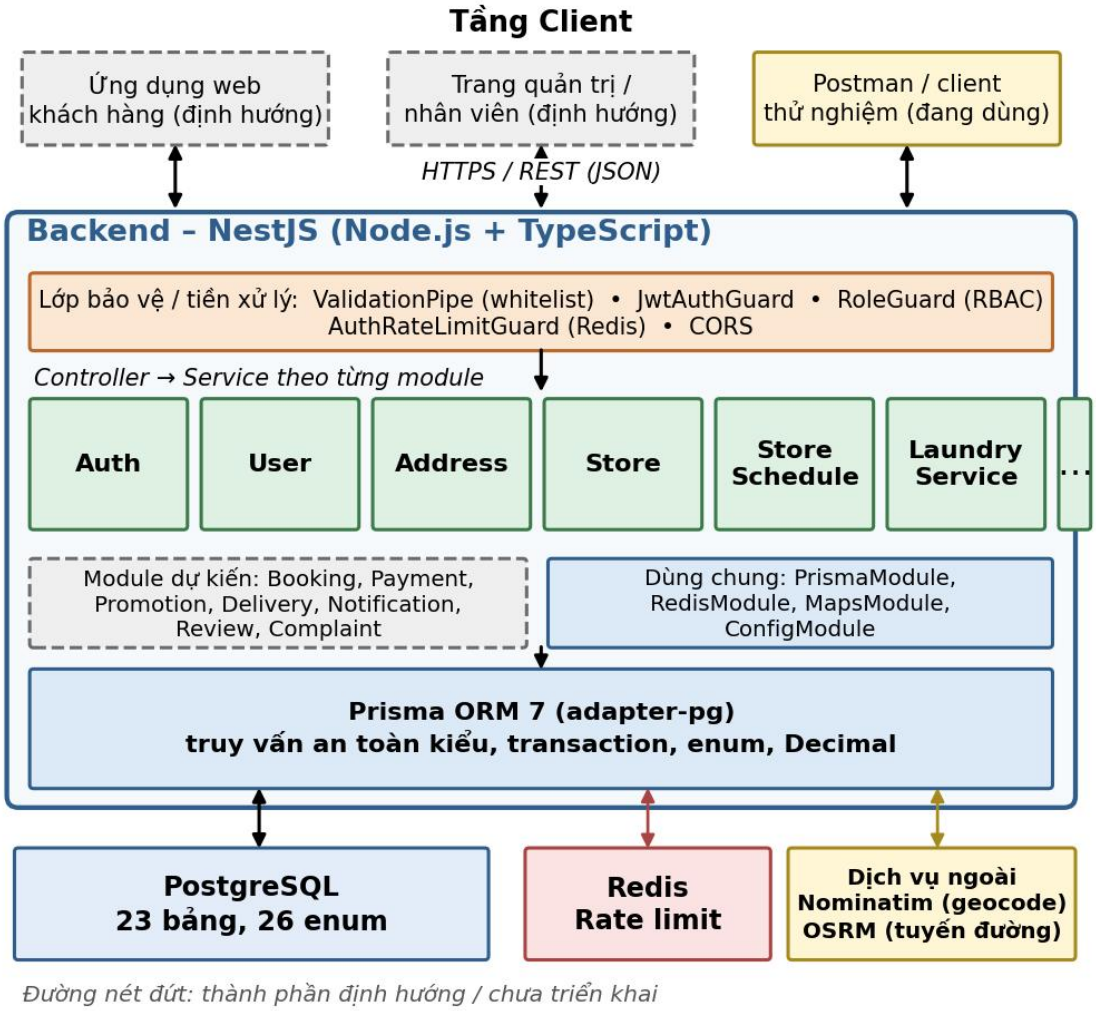
Hình 3.1 mô tả các chức năng chính theo từng tác nhân. Các use case của khách hàng nằm bên trái, các use case của nhân viên, shipper và quản trị viên nằm bên phải, được tô màu theo tác nhân.



Hình 3.1. Sơ đồ use case tổng quát của hệ thống

3.6. Kiến trúc hệ thống

Hệ thống áp dụng kiến trúc nhiều tầng (xem Hình 3.2). Yêu cầu từ client đi qua lớp bảo vệ (kiểm tra dữ liệu, xác thực JWT, phân quyền, giới hạn tần suất), tới controller của từng module, sau đó gọi service để xử lý nghiệp vụ và truy cập dữ liệu thông qua Prisma. Redis chỉ được dùng cho việc đếm giới hạn tần suất; dịch vụ ngoài (Nominatim, OSRM) được gọi qua MapsModule.



Hình 3.2. Kiến trúc tổng quát của hệ thống

Mỗi nghiệp vụ là một module độc lập; các module phụ thuộc nhau theo một chiều để tránh vòng phụ thuộc (Bảng 3.4). Ví dụ `StoreScheduleModule` và `LaundryServiceModule` đều dùng lại việc kiểm tra quyền quản lý cửa hàng do `StoreModule` cung cấp, thay vì sao chép logic.

Bảng 3.4. Các module và trách nhiệm

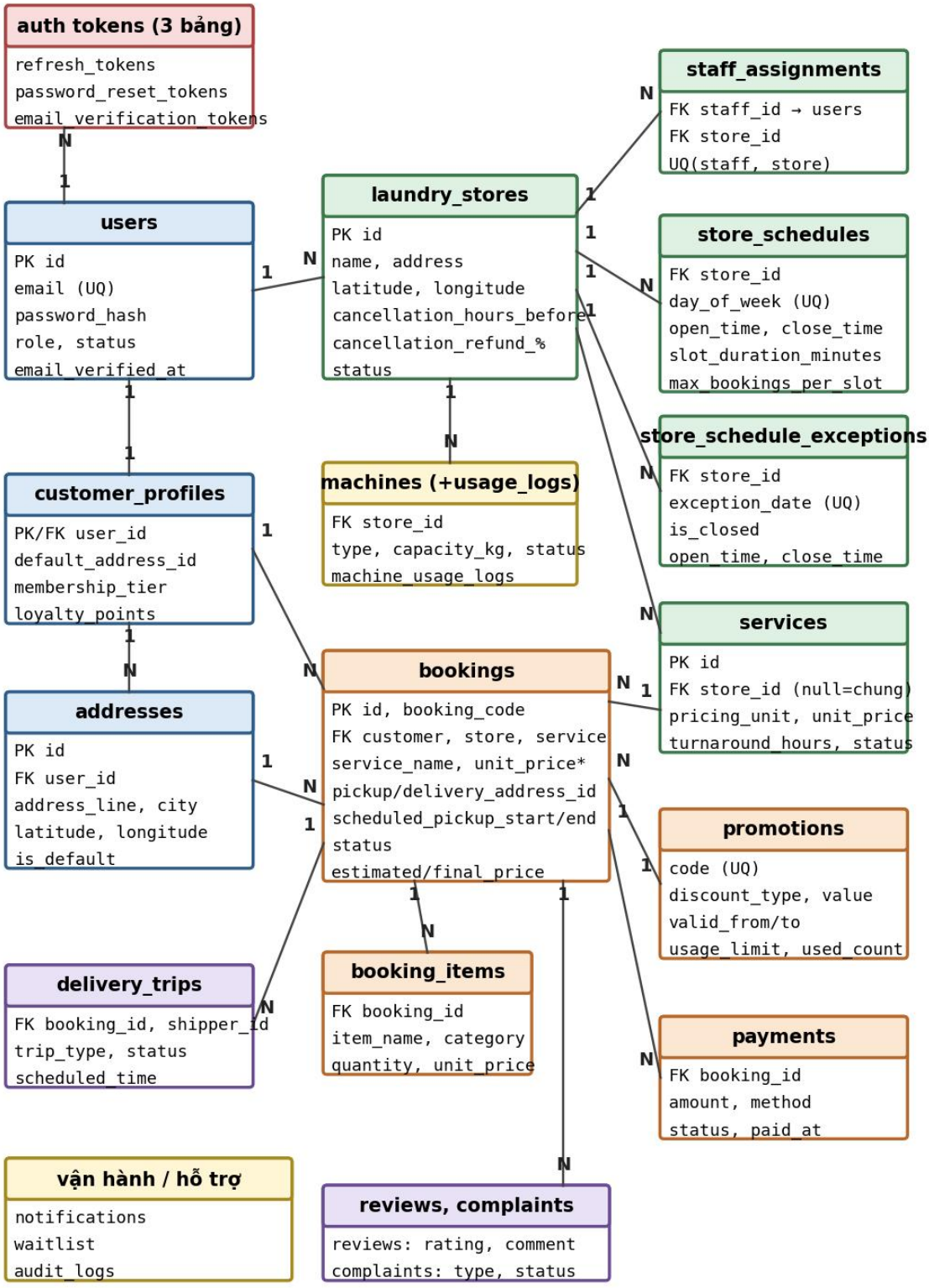
Module	Trách nhiệm chính	Phụ thuộc
AuthModule	Đăng ký, đăng nhập, token, đặt lại mật khẩu; guard JWT, vai trò, giới hạn tần suất	UserModule, các module token
UserModule	Hồ sơ người dùng; cung cấp dịch vụ băm mật khẩu	Prisma
AddressModule	Sổ địa chỉ của người dùng	Prisma, MapsModule
StoreModule	Cửa hàng, phân công nhân viên, kiểm tra quyền quản lý	Prisma, MapsModule
StoreScheduleModule	Lịch tuần, ngày ngoại lệ, sinh khung giờ còn trống	Prisma, StoreModule

Module	Trách nhiệm chính	Phụ thuộc
LaundryServiceModule	Dịch vụ giặt của cửa hàng và dùng chung	Prisma, StoreModule
BookingModule	Đặt lịch, hủy, chốt giá	StoreSchedule, LaundryService, Address, Promotion
MapsModule	Geocoding (Nominatim), tuyến đường (OSRM), khoảng cách Haversine	—
Prisma / Redis / Config	Truy cập CSDL, đếm tần suất, nạp và kiểm tra cấu hình	—

3.7. Thiết kế cơ sở dữ liệu

3.7.1. Mô hình thực thể – quan hệ

Cơ sở dữ liệu gồm **23 bảng** và **26 kiểu enum**, được chia thành các nhóm: người dùng và xác thực, cửa hàng và lịch, dịch vụ, đặt lịch và thanh toán, giao nhận, phản hồi, vận hành và hỗ trợ (Hình 3.3). Để dễ đọc, các bảng ít quan hệ được gộp trong một khung.



*1 – N: một – nhiều; 1 – 1: một – một; * = giá trị chốt tại thời điểm đặt.*
Màu: xanh dương = người dùng/địa chỉ; xanh lá = cửa hàng; cam = đặt lịch/thanh toán; tím = giao nhận/phản hồi; vàng = vận hành/hỗ trợ; đỏ = token xác thực.

Hình 3.3. Sơ đồ thực thể – quan hệ (rút gọn thuộc tính)

3.7.2. Danh sách bảng

Bảng 3.5. Các bảng dữ liệu và trạng thái sử dụng

Bảng	Mô tả	Trạng thái
users	Tài khoản, vai trò (4 loại), trạng thái, thời điểm xác minh email	Đang dùng
refresh_tokens	Refresh token (lưu giá trị băm), hỗ trợ thu hồi	Đang dùng
password_reset_tokens	Token đặt lại mật khẩu dùng một lần	Đang dùng
email_verification_tokens	Token xác minh email dùng một lần	Đang dùng
addresses	Sổ địa chỉ, tọa độ, địa chỉ mặc định	Đang dùng
laundry_stores	Cửa hàng giặt là, chính sách hủy và hoàn tiền	Đang dùng
staff_assignments	Phân công nhân viên – cửa hàng	Đang dùng
store_schedules	Lịch mở cửa theo thứ, độ dài slot, sức chứa mỗi slot	Đang dùng
store_schedule_exceptions	Ngày nghỉ / đổi giờ đặc biệt	Đang dùng
services	Dịch vụ giặt, đơn vị tính, đơn giá, thời gian xử lý	Đang dùng
bookings	Đơn đặt lịch, snapshot dịch vụ và giá, trạng thái	Đã chuẩn bị
booking_items	Các món đồ trong đơn	Đã chuẩn bị
customer_profiles	Tùy chọn giặt, hạng thành viên, điểm tích lũy	Dự kiến
payments	Giao dịch thanh toán, hoàn tiền	Dự kiến
promotions	Mã khuyến mãi, hạn dùng, số lượt	Dự kiến
delivery_trips	Chuyến lấy / giao đồ, shipper, ảnh xác nhận	Dự kiến
machines, machine_usage_logs	Máy giặt/sấy của cửa hàng và nhật ký sử dụng theo đơn	Dự kiến
reviews	Đánh giá 1–5 sao sau đơn	Dự kiến
complaints	Khiếu nại (mất, hỏng, nhầm đồ, giao trễ), xử lý, bồi thường	Dự kiến
notifications	Thông báo theo sự kiện, kênh gửi	Dự kiến
waitlist	Danh sách chờ khi khung giờ đã đầy	Dự kiến
audit_logs	Nhật ký thao tác (giá trị cũ/mới, IP)	Dự kiến

3.7.3. Một số quyết định thiết kế

- **Token lưu băm:** các cột `token_hash` kiểu `CHAR(64)` chứa giá trị HMAC-SHA256 và có ràng buộc duy nhất.
- **Dịch vụ dùng chung:** `services.store_id` cho phép `NULL` – dịch vụ không thuộc cửa hàng nào được dùng cho mọi cửa hàng.
- **Snapshot trong đơn:** `bookings` lưu `service_name`, `pricing_unit`, `unit_price` tại thời điểm đặt (đã bổ sung vào lược đồ), nhờ đó đổi giá hoặc tên dịch vụ không làm sai lệch đơn cũ và việc tính `final_price` theo cân nặng thực tế luôn dùng đúng đơn giá đã báo.

- **Ràng buộc duy nhất:** `(store_id, day_of_week)` cho lịch tuần, `(store_id, exception_date)` cho ngày ngoại lệ, `(staff_id, store_id)` cho phân công – để cơ sở dữ liệu chặn trùng ngay cả khi mã ứng dụng bỏ sót.
- **Xóa mềm:** cửa hàng và dịch vụ chuyển sang `inactive` thay vì xóa, vì đơn đặt lịch cũ vẫn tham chiếu tới chúng.
- **Kiểu dữ liệu:** tiền và tọa độ dùng `DECIMAL`; thời điểm dùng `TIMESTAMP`; trường `updated_at` tự cập nhật nhờ `@updatedAt`.

Từ điển dữ liệu chi tiết của toàn bộ 23 bảng được trình bày ở Phụ lục B.

3.8. Thiết kế API

Hệ thống hiện cung cấp **42 endpoint** thuộc 8 nhóm (Bảng 3.6); danh sách đầy đủ kèm quyền truy cập ở Phụ lục A. Các endpoint đọc dữ liệu công khai (cửa hàng, dịch vụ, lịch, khung giờ) không yêu cầu đăng nhập; mọi thao tác ghi đều yêu cầu xác thực và đúng vai trò.

Bảng 3.6. Tổng hợp nhóm API

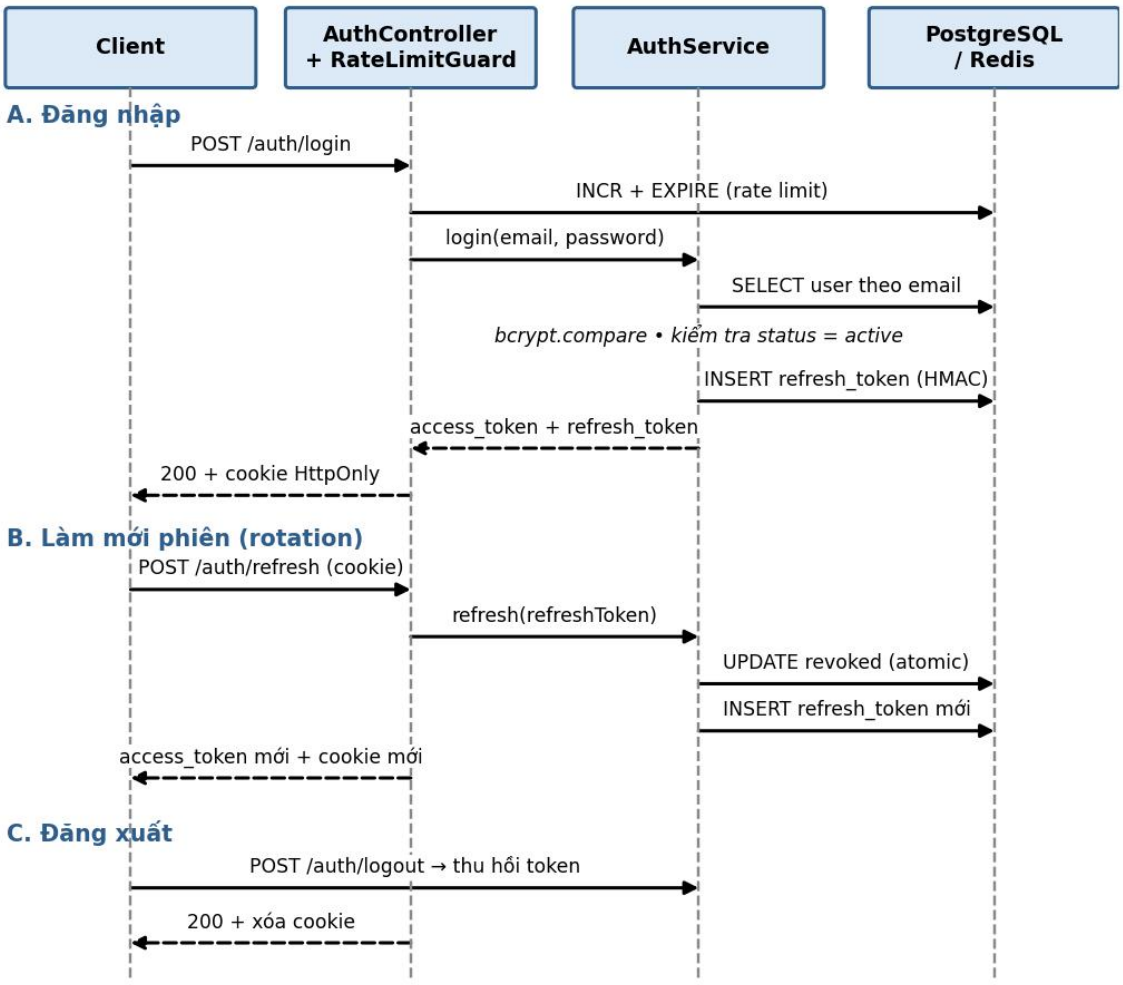
Nhóm	Tiền tố	Số endpoint	Quyền truy cập
Hệ thống	/api	2	Công khai (kiểm tra sống, kiểm tra CSDL)
Xác thực	/api/auth	8	Công khai, có giới hạn tần suất
Người dùng	/api/users	4	Đăng nhập; danh sách/chi tiết chỉ admin
Địa chỉ	/api/addresses	6	Đăng nhập (chỉ dữ liệu của chính mình)
Cửa hàng	/api/stores	8	Đọc công khai; ghi: admin, NV được gán
Lịch & khung giờ	/api/stores/:storeId/…	6	Đọc công khai; ghi: admin, NV được gán
Dịch vụ của cửa hàng	/api/stores/:storeId/services	6	Đọc công khai; ghi: admin, NV được gán
Dịch vụ dùng chung	/api/services	2	Chỉ admin

3.9. Thiết kế các luồng nghiệp vụ chính

3.9.1. Xác thực và quản lý phiên

Hình 3.4 mô tả ba luồng: đăng nhập, làm mới phiên và đăng xuất. Điểm đáng chú ý:

- Mật khẩu sai và email không tồn tại cùng trả thông điệp "Invalid credentials" để không lộ email nào đã đăng ký; chỉ sau khi mật khẩu đúng mới kiểm tra trạng thái tài khoản (`active`).
- Làm mới phiên thu hồi refresh token cũ bằng một câu lệnh cập nhật có điều kiện (`revoked_at IS NULL`), nên hai yêu cầu đồng thời chỉ có một yêu cầu thành công.
- Đăng xuất là thao tác lũy đẳng: token không tồn tại hoặc đã thu hồi vẫn trả thành công.
- Đặt lại mật khẩu đánh dấu token đã dùng trước, rồi mới đổi mật khẩu và thu hồi toàn bộ phiên đăng nhập của người dùng.



Hình 3.4. Sơ đồ trình tự đăng nhập, làm mới phiên và đăng xuất

Các API nhạy cảm được giới hạn tần suất bằng Redis như Bảng 3.7; vượt ngưỡng trả mã `429`.

Bảng 3.7. Chính sách giới hạn tần suất

Endpoint	Tối đa	Cửa sổ	Khóa đếm
POST /auth/register	5 lần	15 phút	IP
POST /auth/login	5 lần	60 giây	IP và email
POST /auth/logout, /auth/refresh	10 lần	60 giây	IP
POST /auth/verify-email, /auth/reset-password	5 lần	15 phút	IP
POST /auth/resend-verification, /auth/forgot-password	3 lần	15 phút	IP và email

3.9.2. Sinh khung giờ còn trống

API `GET /stores/:storeId/available-slots?date=YYYY-MM-DD` trả về các khung giờ của một ngày kèm số chỗ còn lại. Thuật toán gồm các bước:

- Kiểm tra cửa hàng đang hoạt động và ngày hợp lệ, không ở quá khứ (so với giờ địa phương của cửa hàng).
- Tra lịch tuần theo thứ của ngày và tra ngày ngoại lệ (nếu có); xác định giờ mở/đóng theo quy tắc ở Bảng 3.8.
- Chia khoảng [mở, đóng] thành các slot liên tiếp dài `slot_duration_minutes`; slot cuối không đủ độ dài bị bỏ.
- Đếm các đơn của cửa hàng có giờ bắt đầu nằm trong ngày và trạng thái khác `cancelled`, `no_show`, nhóm theo giờ bắt đầu.
- Với mỗi slot: `available = max(sức chứa − số đơn, 0)`; nếu là ngày hôm nay thì loại các slot đã bắt đầu.

Bảng 3.8. Quy tắc ghi đè của ngày ngoại lệ

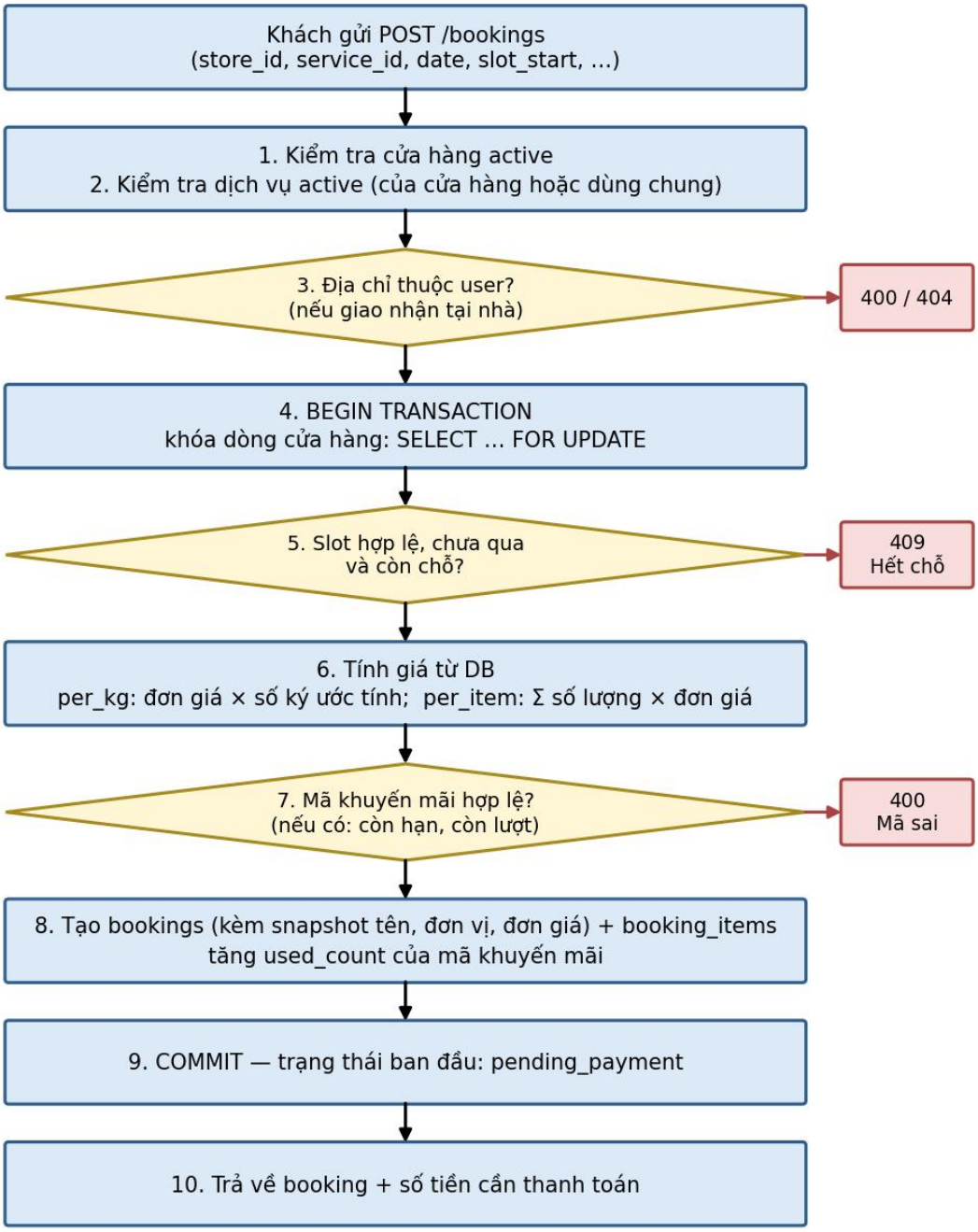
Tình huống	Kết quả
Không có lịch tuần và không có ngoại lệ cho ngày đó	Cửa hàng đóng cửa, không có slot
Có lịch tuần, không có ngoại lệ	Mở theo giờ trong lịch tuần
Ngoại lệ với `is_closed = true`	Đóng cả ngày; trả kèm lý do nghỉ
Ngoại lệ với `is_closed = false` kèm giờ mở/đóng	Dùng giờ của ngoại lệ; độ dài slot và sức chứa lấy từ lịch tuần (mặc định 60 phút và 5 đơn nếu chưa có)

Quy ước múi giờ: các cột `TIMESTAMP` và `TIME` không lưu múi giờ, nên mọi giá trị được hiểu là giờ địa phương của cửa hàng (UTC+7) và lưu nguyên văn, không quy đổi. Cách làm này giúp so khớp giờ bắt đầu của đơn với giờ bắt đầu của slot một cách trực tiếp, và là quy ước bắt buộc khi cài đặt module đặt lịch.

3.9.3. Quy trình đặt lịch

Quy trình tạo đơn (Hình 3.5) được thiết kế để vừa đúng nghiệp vụ vừa an toàn khi nhiều người đặt đồng thời:

- **Chống vượt công suất:** sau các bước kiểm tra nhẹ, hệ thống mở transaction và khóa dòng cửa hàng (`SELECT … FOR UPDATE`). Các đơn của cùng một cửa hàng được xử lý lần lượt, nên việc đếm chỗ và ghi đơn không bị xen kẽ.
- **Giá do máy chủ quyết định:** đơn giá lấy từ cơ sở dữ liệu, không nhận từ client (Bảng 3.9).
- **Snapshot:** tên dịch vụ, đơn vị tính, đơn giá được ghi vào đơn cùng lúc tạo đơn.
- **Nguyên tử:** tạo đơn, tạo món đồ và tăng lượt dùng mã khuyến mãi nằm trong một transaction; lỗi bất kỳ sẽ hoàn tác toàn bộ.



Các bước 4–9 nằm trong cùng một transaction; lỗi ở bất kỳ bước nào sẽ ROLLBACK, không để lại dữ liệu dở dang.

Hình 3.5. Lưu đồ tạo đơn đặt lịch

Bảng 3.9. Cách tính giá theo loại dịch vụ

Loại dịch vụ	Dữ liệu client gửi	Giá ước tính (estimated_price)	Giá cuối (final_price)
per_kg (theo ký)	estimated_weight_kg (0,5 – 100)	unit_price × estimated_weight_kg	actual_weight_kg × unit_price (đã chốt) − discount_amount

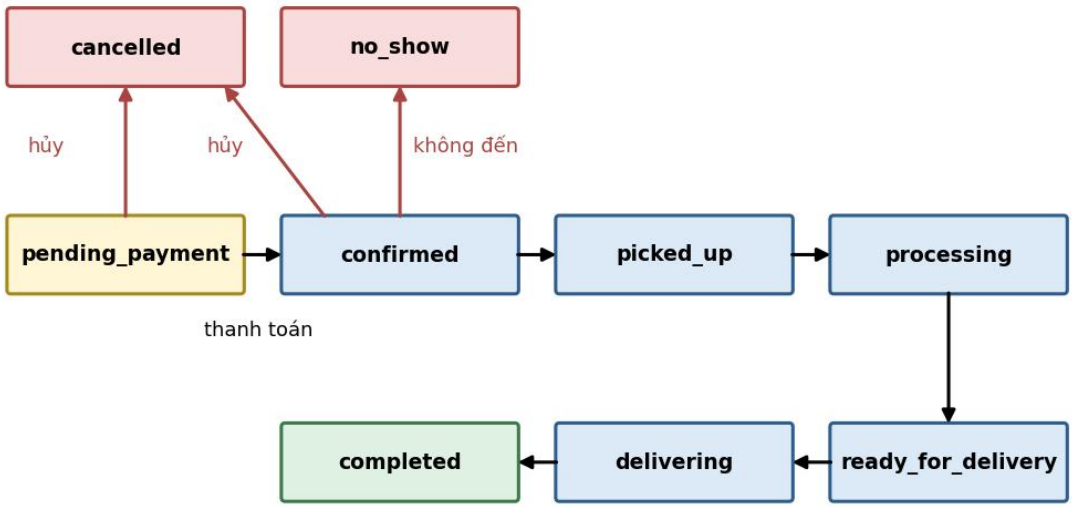
Loại dịch vụ	Dữ liệu client gửi	Giá ước tính (estimated_price)	Giá cuối (final_price)
per_item (theo món)	Danh sách món: tên, loại, số lượng	Σ (số lượng × unit_price)	Σ (số lượng thực tế × unit_price đã chốt) − discount_amount

Khuyến mãi: mã hợp lệ khi đang `active`, nằm trong khoảng `valid_from`–`valid_to` và còn lượt (`used_count < usage_limit`). Mức giảm là phần trăm hoặc số tiền cố định, không vượt quá giá ước tính; số tiền giảm được lưu cố định vào đơn (`discount_amount`) và không tính lại khi cân thực tế. Lượt dùng được tăng bằng câu lệnh cập nhật có điều kiện trong cùng transaction và được hoàn lại khi hủy đơn.

Hủy đơn: chỉ hủy được khi đơn ở trạng thái `pending_payment` hoặc `confirmed`. Hủy trước giờ lấy đồ ít nhất `cancellation_hours_before` giờ thì được hoàn `cancellation_refund_percent`% (theo cấu hình cửa hàng), hủy muộn hơn vẫn được nhưng không hoàn tiền. Đơn đã hủy không còn chiếm chỗ của khung giờ.

3.9.4. Vòng đời trạng thái của đơn

Đơn đi qua các trạng thái như Hình 3.6. Đơn mới tạo ở `pending_payment` và giữ chỗ; sau thanh toán chuyển `confirmed`; từ đây lần lượt đến `picked_up`, `processing`, `ready_for_delivery`, `delivering` và `completed`. Hai nhánh kết thúc bất thường là `cancelled` (khách hủy) và `no_show` (khách không đến khi đã xác nhận).



Các trạng thái lấy từ enum bookings_status_enum; logic chuyển trạng thái thuộc module Booking (đang triển khai).

Hình 3.6. Sơ đồ trạng thái của đơn đặt lịch

3.10. Thiết kế bảo mật

Bảng 3.10. Các mối đe dọa và biện pháp bảo mật

Mối đe dọa	Biện pháp
Lộ cơ sở dữ liệu	Mật khẩu băm bcrypt; token lưu HMAC-SHA256; mọi API đều loại `password_hash` khỏi kết quả trả về.
Dò mật khẩu, gửi yêu cầu hàng loạt	Giới hạn tần suất bằng Redis (mã 429); bcrypt có cost cao; giới hạn độ dài mật khẩu.
Đánh cắp hoặc tái sử dụng refresh token	Cookie HttpOnly + SameSite, rotation, thu hồi nguyên tử, thu hồi toàn bộ phiên khi đặt lại mật khẩu; access token sống ngắn.
Liệt kê tài khoản (user enumeration)	Thông điệp chung cho đăng nhập sai, quên mật khẩu và gửi lại xác minh.
Gán đè trường (mass assignment)	ValidationPipe whitelist + forbidNonWhitelisted; DTO riêng cho người dùng và admin; không cho client sửa role, password_hash.
Leo thang đặc quyền, truy cập dữ liệu người khác	RoleGuard theo vai trò; kiểm tra phân công nhân viên – cửa hàng; mọi truy vấn địa chỉ đều lọc theo `user_id` của người đang đăng nhập.
Đặt trùng, vượt công suất khi đồng thời	Transaction kết hợp khóa dòng; ràng buộc duy nhất ở cơ sở dữ liệu.
SQL injection	Prisma tham số hóa mọi truy vấn; dữ liệu đầu vào được kiểm tra bằng DTO.
Gọi API từ nguồn lạ (CORS)	Chỉ cho phép các origin khai báo trong cấu hình; không dùng ký tự đại diện khi bật credentials.
Lộ bí mật cấu hình	Không có khóa mặc định trong mã; biến môi trường bắt buộc được kiểm tra khi khởi động; tệp `.env` không đưa vào Git.

3.11. Kết luận chương 3

Chương 3 đã phân tích nghiệp vụ, xác định tác nhân và yêu cầu, trình bày kiến trúc nhiều tầng theo module, thiết kế cơ sở dữ liệu 23 bảng, tập API và các luồng nghiệp vụ trọng tâm (xác thực, sinh khung giờ, đặt lịch, vòng đời đơn) cùng các biện pháp bảo mật. Chương tiếp theo trình bày việc cài đặt và kiểm thử.

CHƯƠNG 4: TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1. Giới thiệu chương

Chương này trình bày môi trường phát triển, cấu trúc mã nguồn, việc cài đặt từng module đã hoàn thành, kế hoạch và kết quả kiểm thử, đồng thời đánh giá mức độ đạt được so với mục tiêu ban đầu.

4.2. Môi trường phát triển

Bảng 4.1. Môi trường phát triển

Hạng mục	Giá trị
Hệ điều hành	Windows 11
Runtime / ngôn ngữ	Node.js 24, TypeScript 6 (chế độ strict)
Framework	NestJS 12 (nền tảng Express)
Cơ sở dữ liệu	PostgreSQL (cục bộ, cổng 5432); lược đồ quản lý bằng Prisma 7.10
Bộ nhớ đệm / đếm tần suất	Redis (cục bộ)
Quản lý gói	pnpm
Công cụ	VS Code, Postman, Git; kiểm tra bằng oxlint, kiểm thử bằng Vitest

4.3. Cấu trúc mã nguồn

Mã nguồn được tổ chức theo module nghiệp vụ; mỗi module gồm controller, service, module và thư mục `dto` chứa các lớp kiểm tra dữ liệu đầu vào:

```
src/
├── main.ts                    # khởi động: ValidationPipe, CORS, cookie-parser, tiền tố /api
├── app.module.ts              # gom các module, nạp và kiểm tra cấu hình
├── config/env.validation.ts   # kiểm tra biến môi trường bắt buộc
├── common/decorators/         # @NormalizeEmail
├── prisma/  redis/            # PrismaService, RedisService (dùng chung)
├── utils/maps/                # MapsService (Nominatim, OSRM), haversineKm
├── modules/
│   ├── auth/                  # đăng nhập, token, guard, rate limit
│   │   ├── refresh_token/  password_reset_token/  email_verification_token/
│   ├── user/    address/
│   ├── store/
│   ├── store-schedule/        # lịch tuần, ngoại lệ, sinh slot (schedule-time.util.ts)
│   └── laundry-service/       # dịch vụ của cửa hàng + dịch vụ dùng chung
└── generated/prisma/          # client do Prisma sinh ra (không đưa vào Git)
prisma/schema.prisma           # lược đồ cơ sở dữ liệu (23 bảng, 26 enum)
```

4.4. Cài đặt các module

4.4.1. Khởi tạo ứng dụng và cấu hình

- `ValidationPipe` toàn cục với `whitelist`, `forbidNonWhitelisted`, `transform`: mọi DTO đều được kiểm tra, trường lạ bị từ chối.
- `validateEnv` dừng ứng dụng ngay khi thiếu một trong các biến bắt buộc (chuỗi kết nối CSDL, các khóa bí mật JWT/token/Redis); không còn khóa

mặc định trong mã. `JwtModule` được đăng ký bất đồng bộ để đọc khóa sau khi `.env` đã nạp.

- CORS chỉ cho phép các origin khai báo trong biến `CORS_ORIGIN` và bật `credentials` để trình duyệt gửi cookie refresh token.

4.4.2. Module xác thực và người dùng

Đây là module được rà soát kỹ nhất về bảo mật. Các điểm cài đặt chính:

- Đăng ký: chuẩn hóa email (cắt khoảng trắng, chữ thường), kiểm tra trùng, băm mật khẩu bằng bcrypt, tạo token xác minh email (32 byte ngẫu nhiên, hiệu lực 15 phút). Chưa có dịch vụ gửi thư nên ở môi trường phát triển token được ghi ra nhật ký máy chủ (tắt hoàn toàn khi `NODE_ENV=production`).
- Đăng nhập: so khớp bcrypt, kiểm tra tài khoản `active`, cấp access token (JWT, mặc định 15 phút) và refresh token đặt trong cookie `HttpOnly`.
- Token lưu băm: cả ba loại token đều tra cứu và đánh dấu đã dùng bằng giá trị HMAC-SHA256, và các thao tác "dùng một lần" là nguyên tử.
- Giới hạn tần suất bằng `AuthRateLimitGuard` (Redis), trả `429` khi vượt ngưỡng.
- Người dùng: mọi API trả về chỉ gồm các trường an toàn (không có `password_hash`); `GET /users`, `GET /users/:id` chỉ dành cho admin; người dùng tự sửa hồ sơ qua `PATCH /users/me` với DTO chỉ gồm họ tên, số điện thoại và ảnh đại diện.

Đoạn mã băm token và thu hồi refresh token nguyên tử (rút gọn):

```
private hashToken(token: string): string {
  const secret = process.env.JWT_REFRESH_SECRET;
  if (!secret) throw new InternalServerErrorException('JWT_REFRESH_SECRET is not configured');
  return createHmac('sha256', secret).update(token).digest('hex');
}


async revoke(token: string): Promise<boolean> {
  const { count } = await this.prisma.refresh_tokens.updateMany({
    where: { token_hash: this.hashToken(token), revoked_at: null },
    data: { revoked: true, revoked_at: new Date() },
  });
  return count > 0;   // false nếu token đã bị thu hồi trước đó
}
```

4.4.3. Module địa chỉ

- Mỗi người dùng chỉ truy cập được địa chỉ của chính mình (mọi truy vấn lọc theo `user_id`); địa chỉ của người khác trả `404`.
- Địa chỉ đầu tiên luôn là mặc định. Khi đặt địa chỉ khác làm mặc định, việc bỏ cờ cũ và đặt cờ mới nằm trong một transaction để luôn có tối đa một địa chỉ mặc định. Khi xóa địa chỉ mặc định, địa chỉ cũ nhất còn lại được đặt làm mặc định.

- Tọa độ do máy chủ tự xác định bằng Nominatim; lỗi dịch vụ bản đồ chỉ ghi cảnh báo và địa chỉ vẫn được lưu với tọa độ trống. Khi sửa nội dung địa chỉ, tọa độ cũ bị bỏ và được tính lại.
- Xóa địa chỉ đang được đơn đặt lịch tham chiếu bị chặn bởi khóa ngoại và trả `409` thay vì lỗi máy chủ.

4.4.4. Module cửa hàng

- Danh sách công khai chỉ gồm cửa hàng `active`, hỗ trợ tìm theo tên/địa chỉ (không phân biệt hoa thường), phân trang và sắp xếp theo khoảng cách khi có `lat`, `lng` (kết quả kèm `distance_km`, cửa hàng chưa có tọa độ xếp cuối).
- Phân quyền hai tầng: `RoleGuard` kiểm tra vai trò, sau đó service kiểm tra quan hệ phân công trong `staff_assignments`. Nhân viên được sửa cửa hàng mình phụ trách nhưng không được đổi trạng thái; chỉ admin tạo, vô hiệu hóa cửa hàng và gán/gỡ nhân viên.
- Tạo hoặc đổi địa chỉ cửa hàng sẽ tự tính tọa độ; admin có thể gửi tọa độ thủ công (bắt buộc đủ cặp vĩ độ, kinh độ).
- Vô hiệu hóa là xóa mềm (`status = inactive`); cửa hàng đã vô hiệu hóa không còn xuất hiện ở API công khai.

4.4.5. Module lịch mở cửa và khung giờ

- `PUT /stores/:storeId/schedules` thay thế toàn bộ lịch tuần trong một transaction: các ngày gửi lên được `upsert`, ngày còn lại bị xóa (nghỉ cả ngày). Kiểm tra giờ mở trước giờ đóng, độ dài slot không dài hơn thời gian mở cửa, không trùng thứ.
- Ngày ngoại lệ được tạo, liệt kê và xóa riêng; trùng ngày trả `409`.
- Toàn bộ xử lý giờ và chia slot nằm trong các hàm thuần ở `schedule-time.util.ts` nên kiểm thử được độc lập với cơ sở dữ liệu.

Hàm sinh khung giờ:

```
export function generateSlots(open: string, close: string, slotMinutes: number): TimeSlot[] {
  if (slotMinutes <= 0) return [];
  const slots: TimeSlot[] = [];
  const closeMinutes = toMinutes(close);
  for (let start = toMinutes(open); start + slotMinutes <= closeMinutes; start += slotMinutes) {
    slots.push({ start: fromMinutes(start), end: fromMinutes(start + slotMinutes) });
  }
  return slots;   // slot cuối không đủ độ dài sẽ bị bỏ
}
```

4.4.6. Module dịch vụ giặt

- Dịch vụ riêng của cửa hàng: thêm, sửa, ngừng (xóa mềm); admin hoặc nhân viên được gán mới được thao tác. Tên dịch vụ không trùng trong cùng cửa hàng (không phân biệt hoa thường).

- API công khai trả các dịch vụ `active` của cửa hàng cộng với dịch vụ dùng chung (`store_id = NULL`), mỗi dịch vụ có trường `scope` là `store` hoặc `global`.
- Dịch vụ dùng chung có hai API riêng chỉ dành cho admin (`GET /services`, `PATCH /services/:id`); sửa dịch vụ của cửa hàng qua đường dẫn này trả `404` để tránh sửa nhầm.
- Giá trả về dạng số (không phải chuỗi `Decimal`) và được kiểm tra: lớn hơn 0, tối đa hai chữ số thập phân.

4.4.7. Chuẩn bị cho module đặt lịch

Để đáp ứng quy tắc R4 (chốt giá tại thời điểm đặt), lược đồ `bookings` đã được bổ sung ba cột và đồng bộ lên cơ sở dữ liệu bằng `prisma db push` (bảng đang trống nên cột bắt buộc được thêm an toàn; lệnh `prisma migrate diff` được dùng trước để xác nhận câu lệnh chỉ thêm ba cột):

```
service_name  String                      @db.VarChar(100)
pricing_unit  services_pricing_unit_enum
unit_price    Decimal                     @db.Decimal(10, 2)
```

Module đặt lịch sẽ dùng `StoreScheduleService` để kiểm tra khung giờ, `LaundryServiceService` để lấy đơn giá, `AddressService` để kiểm tra địa chỉ, và thực hiện toàn bộ trong một transaction theo thiết kế ở mục 3.9.3.

4.5. Tiến độ triển khai

Bảng 4.2. Tiến độ triển khai theo module

Module	Endpoint	Kiểm tra tự động	Trạng thái
Auth	8	tsc, lint, khởi động ứng dụng	Hoàn thành
User	4	tsc, lint	Hoàn thành
Address	6	tsc, lint	Hoàn thành
Store	8	tsc, lint, khởi động ứng dụng	Hoàn thành
Store Schedule	6	tsc, lint, 9 unit test (hàm thuần)	Hoàn thành
Laundry Service	8	tsc, lint, kiểm tra 401 khi không có token	Hoàn thành
Booking	4 (dự kiến)	—	Đã thiết kế, đang triển khai
Payment, Promotion, Delivery, Notification, Review, Complaint	—	—	Định hướng

4.6. Kiểm thử

4.6.1. Chiến lược kiểm thử

- **Kiểm tra tĩnh:** biên dịch TypeScript ở chế độ `strict` và chạy oxlint để phát hiện lỗi kiểu, biến thừa, mã không an toàn.
- **Kiểm thử đơn vị:** áp dụng cho các hàm thuần (xử lý giờ, sinh slot) bằng Vitest, không cần cơ sở dữ liệu.
- **Kiểm thử chức năng API:** thực hiện theo các kịch bản ở mục 4.6.3 bằng Postman trên cơ sở dữ liệu cục bộ.

4.6.2. Kết quả kiểm tra tự động

Bảng 4.3. Kết quả kiểm tra tự động

Hạng mục	Cách thực hiện	Kết quả
Biên dịch TypeScript (strict)	tsc --noEmit	Không có lỗi
Phân tích tĩnh	oxlint trên các module store, store-schedule, laundry-service	Không có cảnh báo
Kiểm thử đơn vị	vitest — schedule-time.util.spec.ts	9/9 kiểm thử đạt (sinh slot, đổi giờ, kiểm tra ngày, giờ cửa hàng)
Đóng gói và khởi động	nest build; chạy dist/main.js	Thành công; các route của module được ánh xạ đầy đủ
Kiểm tra quyền truy cập	GET/PATCH /api/services không có token	Cả hai trả 401
Đồng bộ lược đồ	prisma db push + đối chiếu information_schema	Ba cột snapshot đã có trong bảng bookings

4.6.3. Kịch bản kiểm thử chức năng API

Các kịch bản dưới đây được soạn theo từng module. Cột "Đạt" để đánh dấu sau khi chạy thử trên môi trường thực tế. Token xác minh email và đặt lại mật khẩu lấy từ nhật ký máy chủ (môi trường phát triển).

Bảng 4.4. Kịch bản kiểm thử module Auth

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
AUTH-01	Đăng ký hợp lệ	Email mới, mật khẩu Passw0rdOK, tên hợp lệ	201; trả id và email; ghi token xác minh ra nhật ký	✓
AUTH-02	Đăng ký trùng email	Email đã tồn tại	409 Email already exists	✓
AUTH-03	Đăng ký mật khẩu yếu	Mật khẩu "abc"	400, nêu quy tắc mật khẩu	✓
AUTH-04	Đăng ký có trường lạ	Thêm role: admin	400 (forbidNonWhitelisted)	✓
AUTH-05	Xác minh email	Token đúng từ nhật ký	200; email_verified_at được ghi	✓
AUTH-06	Dùng lại token xác minh	Cùng token lần hai	401 token đã được sử dụng	✓

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
AUTH-07	Đăng nhập đúng	Email, mật khẩu đúng	200; access_token; cookie refresh_token HttpOnly	✓
AUTH-08	Đăng nhập sai mật khẩu	Mật khẩu sai	401 Invalid credentials	✓
AUTH-09	Vượt giới hạn đăng nhập	>5 lần trong 60 giây	429	✓
AUTH-10	Làm mới phiên	Cookie refresh hợp lệ	200; access token và cookie mới; token cũ bị thu hồi	✓
AUTH-11	Làm mới bằng token đã thu hồi	Cookie cũ	401	✓
AUTH-12	Đặt lại mật khẩu	Token từ forgot-password	200; các phiên đăng nhập cũ bị thu hồi	✓

Bảng 4.5. Kịch bản kiểm thử module User và Address

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
USER-01	Xem hồ sơ của tôi	GET /users/me có token	200; không chứa password_hash	✓
USER-02	Sửa hồ sơ	PATCH /users/me {full_name, phone}	200; dữ liệu được cập nhật	✓
USER-03	Customer xem danh sách người dùng	GET /users bằng tài khoản customer	403	✓
USER-04	Không có token	GET /users/me không token	401	✓
ADDR-01	Tạo địa chỉ đầu tiên	POST /addresses không gửi is_default	201; is_default = true; có tọa độ nếu geocode được	✓
ADDR-02	Tạo địa chỉ mặc định mới	Địa chỉ thứ hai với is_default = true	201; địa chỉ cũ không còn mặc định	✓
ADDR-03	Đặt địa chỉ mặc định	PATCH /addresses/:id/default	200; chỉ còn một địa chỉ mặc định	✓
ADDR-04	Xem địa chỉ của người khác	GET /addresses/:id của user khác	404	✓
ADDR-05	Xóa địa chỉ mặc định	DELETE địa chỉ mặc định	200; địa chỉ cũ nhất trở thành mặc định	☐
ADDR-06	Thiếu trường bắt buộc	Không có recipient_name	400	☐

Bảng 4.6. Kịch bản kiểm thử module Store

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
STORE-01	Admin tạo cửa hàng	POST /stores bằng admin	201; tự tính tọa độ; ghi created_by	✓
STORE-02	Customer tạo cửa hàng	POST /stores bằng customer	403	✓
STORE-03	Danh sách công khai	GET /stores không token	200; chỉ cửa hàng active; có meta phân trang	✓
STORE-04	Sắp xếp theo khoảng cách	GET /stores?lat=..&lng=..	Kết quả tăng dần theo distance_km	☐

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
STORE-05	Chỉ gửi lat	GET /stores?lat=..	400	☐
STORE-06	Nhân viên sửa cửa hàng được gán	PATCH /stores/:id bằng staff được gán	200	✓
STORE-07	Nhân viên sửa cửa hàng khác	PATCH bằng staff chưa được gán	403	✓
STORE-08	Nhân viên đổi trạng thái	PATCH {status: inactive} bằng staff	403	✓
STORE-09	Vô hiệu hóa cửa hàng	DELETE /stores/:id bằng admin	200; GET công khai sau đó trả 404	✓
STORE-10	Gán nhân viên trùng	POST /stores/:id/staff lần hai	409	✓

Bảng 4.7. Kịch bản kiểm thử module lịch mở cửa và dịch vụ

Mã	Kịch bản	Điều kiện / dữ liệu	Kết quả mong đợi	Đạt
SCH-01	Đặt lịch tuần	PUT schedules với 5 ngày	200; trả đủ 5 ngày; ngày không gửi bị xóa	✓
SCH-02	Giờ mở sau giờ đóng	open_time ≥ close_time	400	✓
SCH-03	Trùng thứ	Hai phần tử cùng day_of_week	400	✓
SCH-04	Khung giờ ngày thường	GET available-slots ngày có lịch	Danh sách slot đúng độ dài, chia đều từ giờ mở	✓
SCH-05	Ngày nghỉ	Ngoại lệ is_closed = true cho ngày đó	is_open = false, slots rỗng, có lý do	✓
SCH-06	Ngày quá khứ	date nhỏ hơn hôm nay	400	✓
SCH-07	Ngoại lệ trùng ngày	POST schedule-exceptions lần hai	409	✓
SRV-01	Tạo dịch vụ	POST /stores/:id/services	201; unit_price dạng số	✓
SRV-02	Trùng tên trong cửa hàng	Tên đã có (khác hoa/thường)	409	✓
SRV-03	Giá không hợp lệ	unit_price = 0 hoặc âm	400	✓
SRV-04	Danh sách công khai	GET /stores/:id/services	Chỉ dịch vụ active; gồm cả dịch vụ dùng chung có scope = global	✓
SRV-05	Ngừng dịch vụ	DELETE dịch vụ	200; biến mất khỏi API công khai, còn ở /manage	✓
SRV-06	Admin sửa dịch vụ dùng chung	PATCH /services/:id (store_id NULL)	200	✓
SRV-07	Sửa dịch vụ cửa hàng qua /services	PATCH /services/:id (có store_id)	404	✓
SRV-08	Customer gọi API dịch vụ dùng chung	GET /services bằng customer	403	✓

4.7. Đánh giá kết quả đạt được

Bảng 4.8. Đối chiếu kết quả với mục tiêu

Mục tiêu	Mức đạt	Minh chứng
Phân tích nghiệp vụ, tác nhân, quy trình	Đạt	Mục 3.1–3.5
Thiết kế CSDL (23 bảng) và kiến trúc module	Đạt	Mục 3.6–3.7, Phụ lục B
API xác thực, phân quyền, người dùng	Đã cài đặt	12 endpoint; mục 4.4.2
API địa chỉ, cửa hàng, lịch/khung giờ, dịch vụ	Đã cài đặt	28 endpoint; mục 4.4.3–4.4.6
Quy trình đặt lịch chống vượt công suất, chốt giá	Đã thiết kế; đang cài đặt	Mục 3.9.3; lược đồ đã bổ sung snapshot
Bảo mật dữ liệu người dùng	Đạt ở phạm vi đã cài đặt	Bảng 3.10; mục 4.4.2
Kiểm thử và tài liệu	Một phần	9 kiểm thử đơn vị đạt; kịch bản API đã soạn, cần chạy và ghi kết quả

4.8. Hạn chế của hệ thống

- Module đặt lịch chưa được cài đặt, nên luồng đặt lịch end-to-end, thanh toán và xử lý đơn chưa vận hành.
- Chưa gửi được email thật (xác minh email, đặt lại mật khẩu); đang dùng nhật ký máy chủ ở môi trường phát triển.
- Chưa có giao diện người dùng; kiểm thử API hiện chủ yếu thực hiện thủ công, chưa có kiểm thử tích hợp tự động với cơ sở dữ liệu.
- Đơn ở trạng thái chờ thanh toán giữ chỗ không thời hạn cho tới khi có cơ chế tự hủy khi quá hạn thanh toán.
- Lược đồ cơ sở dữ liệu đang đồng bộ bằng `prisma db push`, chưa có lịch sử migration phục vụ triển khai nhiều môi trường.
- Phụ thuộc dịch vụ bản đồ miễn phí (giới hạn 1 yêu cầu/giây); khoảng cách tính theo đường chim bay.
- Khuyến mãi chưa ghi nhận ai đã dùng mã nào nên chưa giới hạn được số lần dùng trên mỗi người.

4.9. Kết luận chương 4

Chương 4 đã trình bày việc cài đặt 6 module nền tảng gồm 40 endpoint nghiệp vụ, cùng kết quả kiểm tra tự động và bộ kịch bản kiểm thử chức năng. Phần lõi dữ liệu và an toàn thông tin đã được hoàn thiện; module đặt lịch đã sẵn sàng về thiết kế và lược đồ để triển khai tiếp.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình thực hiện đề tài "Hệ thống đặt lịch giặt là trực tuyến", tôi đã hoàn thành các nội dung chính:

- Phân tích nghiệp vụ, xác định 4 nhóm tác nhân và 7 quy tắc nghiệp vụ cốt lõi của quy trình đặt lịch giặt là.
- Thiết kế cơ sở dữ liệu quan hệ 23 bảng, kiến trúc backend theo module và tập API REST thống nhất.
- Cài đặt phần nền tảng gồm xác thực – phân quyền, người dùng, địa chỉ, cửa hàng, lịch mở cửa với khung giờ còn trống và dịch vụ – bảng giá, với các cơ chế bảo mật hiện đại (băm mật khẩu và token, refresh token xoay vòng, giới hạn tần suất, kiểm tra dữ liệu chặt).
- Thiết kế quy trình đặt lịch bảo đảm không vượt công suất và chốt giá tại thời điểm đặt, đồng thời chuẩn bị lược đồ dữ liệu cho việc này.

Thông qua đề tài, tôi nắm vững hơn cách tổ chức một ứng dụng backend theo module, thiết kế API và cơ sở dữ liệu, xử lý các vấn đề an toàn (xác thực, phân quyền, tấn công gán đè trường, liệt kê tài khoản) và vấn đề đồng thời (giữ chỗ, thao tác nguyên tử); đồng thời rèn kỹ năng rà soát mã, viết kiểm thử và làm việc với tài liệu kỹ thuật.

5.2. Hạn chế của đề tài

Các hạn chế chính đã nêu ở mục 4.8: module đặt lịch và các chức năng phía sau (thanh toán, giao nhận, thông báo, đánh giá, khiếu nại) chưa được cài đặt; chưa có giao diện người dùng và email thật; kiểm thử tích hợp tự động còn thiếu.

5.3. Hướng phát triển

Định hướng hoàn thiện hệ thống theo thứ tự ưu tiên:

- **Module đặt lịch:** tạo, xem, hủy đơn theo thiết kế ở mục 3.9.3; bổ sung tác vụ nền tự hủy đơn quá hạn thanh toán.
- **Thanh toán và khuyến mãi:** tích hợp VNPay, MoMo, Stripe và thanh toán tiền mặt; quản lý mã giảm giá cho admin; xử lý hoàn tiền theo chính sách hủy.
- **Vận hành cửa hàng:** nhân viên cập nhật trạng thái xử lý đơn; quản lý máy giặt/sấy và nhật ký sử dụng; danh sách chờ khi khung giờ đầy.
- **Giao nhận:** chuyến lấy/giao đồ cho shipper, ảnh xác nhận, tính phí theo quãng đường bằng OSRM.

- **Thông báo và phản hồi:** gửi email/đẩy thông báo theo sự kiện; đánh giá cửa hàng; xử lý khiếu nại và bồi thường; hồ sơ khách hàng, hạng thành viên và tích điểm.
- **Giao diện:** ứng dụng web/di động cho khách hàng, trang quản trị cho nhân viên và admin.
- **Chất lượng và triển khai:** kiểm thử tích hợp tự động, chuyển sang `prisma migrate`, đóng gói Docker, thiết lập CI/CD, ghi nhật ký thao tác (audit log) và giám sát.

5.4. Kết lời

Đề tài đã xây dựng được nền tảng vững chắc về dữ liệu và an toàn thông tin cho một hệ thống đặt lịch giặt là, đồng thời có thiết kế rõ ràng cho các chức năng tiếp theo. Tôi xin chân thành cảm ơn thầy cô đã hướng dẫn và góp ý trong suốt quá trình thực hiện; mọi ý kiến đóng góp của thầy cô sẽ giúp đề tài ngày càng hoàn thiện.

TÀI LIỆU THAM KHẢO

[1] NestJS, "NestJS Documentation". [Trực tuyến]. https://docs.nestjs.com (truy cập 10/2026).

[2] Prisma, "Prisma ORM Documentation". [Trực tuyến]. https://www.prisma.io/docs (truy cập 10/2026).

[3] PostgreSQL Global Development Group, "PostgreSQL Documentation". [Trực tuyến]. https://www.postgresql.org/docs/ (truy cập 10/2026).

[4] Redis Ltd., "Redis Documentation". [Trực tuyến]. https://redis.io/docs/ (truy cập 10/2026).

[5] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015. https://datatracker.ietf.org/doc/html/rfc7519

[6] H. Krawczyk, M. Bellare, R. Canetti, "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, IETF, 1997. https://datatracker.ietf.org/doc/html/rfc2104

[7] N. Provos, D. Mazières, "A Future-Adaptable Password Scheme", Proceedings of the USENIX Annual Technical Conference, 1999.

[8] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures", Luận án tiến sĩ, University of California, Irvine, 2000.

[9] OWASP Foundation, "Authentication Cheat Sheet". [Trực tuyến]. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (truy cập 10/2026).

[10] OWASP Foundation, "Mass Assignment Cheat Sheet". [Trực tuyến]. https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html (truy cập 10/2026).

[11] Nominatim, "Search API" và OpenStreetMap Foundation, "Nominatim Usage Policy". https://nominatim.org/release-docs/latest/api/Search/ ; https://operations.osmfoundation.org/policies/nominatim/ (truy cập 10/2026).

[12] Project OSRM, "Open Source Routing Machine". [Trực tuyến]. https://project-osrm.org (truy cập 10/2026).

[13] R. W. Sinnott, "Virtues of the Haversine", Sky and Telescope, 68(2), tr. 159, 1984.

[14] TypeStack, "class-validator". [Trực tuyến]. https://github.com/typestack/class-validator (truy cập 10/2026).

[15] Vitest, "Vitest Documentation". [Trực tuyến]. https://vitest.dev (truy cập 10/2026).

PHỤ LỤC

PHỤ LỤC A. DANH SÁCH API

Các đường dẫn đều có tiền tố `/api`. "NV được gán" là nhân viên giặt là đã được admin phân công vào cửa hàng tương ứng.

Phương thức	Đường dẫn	Quyền	Mô tả
GET	/api	Công khai	Kiểm tra máy chủ hoạt động
GET	/api/db-health	Công khai	Kiểm tra kết nối cơ sở dữ liệu
POST	/api/auth/register	Công khai	Đăng ký tài khoản
POST	/api/auth/login	Công khai	Đăng nhập, cấp access token và cookie refresh token
POST	/api/auth/logout	Cookie refresh	Đăng xuất, thu hồi refresh token
POST	/api/auth/refresh	Cookie refresh	Làm mới phiên (xoay vòng refresh token)
POST	/api/auth/verify-email	Công khai	Xác minh email bằng token
POST	/api/auth/resend-verification	Công khai	Gửi lại token xác minh email
POST	/api/auth/forgot-password	Công khai	Yêu cầu đặt lại mật khẩu
POST	/api/auth/reset-password	Công khai	Đặt lại mật khẩu bằng token
GET	/api/users/me	Đăng nhập	Xem hồ sơ của tôi
PATCH	/api/users/me	Đăng nhập	Sửa hồ sơ (họ tên, số điện thoại, ảnh đại diện)
GET	/api/users	Admin	Danh sách người dùng
GET	/api/users/:id	Admin	Chi tiết người dùng
POST	/api/addresses	Đăng nhập	Tạo địa chỉ
GET	/api/addresses	Đăng nhập	Danh sách địa chỉ của tôi
GET	/api/addresses/:id	Đăng nhập	Chi tiết địa chỉ
PATCH	/api/addresses/:id	Đăng nhập	Sửa địa chỉ
PATCH	/api/addresses/:id/default	Đăng nhập	Đặt địa chỉ mặc định
DELETE	/api/addresses/:id	Đăng nhập	Xóa địa chỉ
GET	/api/stores	Công khai	Danh sách cửa hàng active (tìm kiếm, phân trang, khoảng cách)
GET	/api/stores/manage	Admin, NV	Cửa hàng quản lý được (NV: cửa hàng được gán)
GET	/api/stores/:id	Công khai	Chi tiết cửa hàng active
POST	/api/stores	Admin	Tạo cửa hàng
PATCH	/api/stores/:id	Admin, NV được gán	Sửa cửa hàng (NV không đổi được status)
DELETE	/api/stores/:id	Admin	Vô hiệu hóa cửa hàng (xóa mềm)
POST	/api/stores/:id/staff	Admin	Gán nhân viên vào cửa hàng
DELETE	/api/stores/:id/staff/:staffId	Admin	Gỡ nhân viên khỏi cửa hàng

Phương thức	Đường dẫn	Quyền	Mô tả
GET	/api/stores/:storeId/schedules	Công khai	Lịch mở cửa theo tuần
PUT	/api/stores/:storeId/schedules	Admin, NV được gán	Thay toàn bộ lịch tuần
GET	/api/stores/:storeId/schedule-exceptions	Công khai	Danh sách ngày ngoại lệ
POST	/api/stores/:storeId/schedule-exceptions	Admin, NV được gán	Tạo ngày ngoại lệ
DELETE	/api/stores/:storeId/schedule-exceptions/:exceptionId	Admin, NV được gán	Xóa ngày ngoại lệ
GET	/api/stores/:storeId/available-slots?date=	Công khai	Khung giờ còn trống trong ngày
GET	/api/stores/:storeId/services	Công khai	Dịch vụ active (của cửa hàng và dùng chung)
GET	/api/stores/:storeId/services/manage	Admin, NV được gán	Dịch vụ của cửa hàng, gồm cả inactive
GET	/api/stores/:storeId/services/:serviceId	Công khai	Chi tiết dịch vụ
POST	/api/stores/:storeId/services	Admin, NV được gán	Tạo dịch vụ
PATCH	/api/stores/:storeId/services/:serviceId	Admin, NV được gán	Sửa dịch vụ
DELETE	/api/stores/:storeId/services/:serviceId	Admin, NV được gán	Ngừng dịch vụ (xóa mềm)
GET	/api/services	Admin	Danh sách dịch vụ dùng chung
PATCH	/api/services/:serviceId	Admin	Sửa dịch vụ dùng chung

API dự kiến của module đặt lịch (đang triển khai)

Phương thức	Đường dẫn	Quyền	Mô tả
POST	/api/bookings	Customer	Tạo đơn đặt lịch
GET	/api/bookings/my	Customer	Danh sách đơn của tôi
GET	/api/bookings/:id	Chủ đơn, admin, NV được gán	Chi tiết đơn
POST	/api/bookings/:id/cancel	Chủ đơn	Hủy đơn, trả tỷ lệ hoàn tiền

PHỤ LỤC B. TỪ ĐIỂN DỮ LIỆU

Từ điển dữ liệu được trích tự động từ tệp `prisma/schema.prisma` (23 bảng, 237 cột), bảo đảm khớp với lược đồ đang dùng.

B.1. users

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
`id`	INT	Không	PK; mặc định: tự tăng
`full_name`	VARCHAR(100)	Không	
`email`	VARCHAR(150)	Không	UNIQUE

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
password_hash	VARCHAR(255)	Không	
phone	VARCHAR(20)	Có	
role	ENUM(admin, laundry_staff, shipper, customer)	Không	mặc định: customer
avatar_url	VARCHAR(255)	Có	
status	ENUM(active, inactive, banned)	Không	mặc định: active
email_verified_at	TIMESTAMP(6)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.2. refresh_tokens

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
user_id	INT	Không	FK → users.id
token_hash	VARCHAR(255)	Không	UNIQUE
expires_at	TIMESTAMP(6)	Không	
revoked	BOOLEAN	Không	mặc định: false
created_at	TIMESTAMP(6)	Không	mặc định: now()
revoked_at	TIMESTAMP(6)	Có	

B.3. password_reset_tokens

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
user_id	INT	Không	FK → users.id
token_hash	CHAR(64)	Không	UNIQUE
expires_at	TIMESTAMP(6)	Không	
used	BOOLEAN	Không	mặc định: false
created_at	TIMESTAMP(6)	Không	mặc định: now()
used_at	TIMESTAMP(6)	Có	

B.4. email_verification_tokens

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
user_id	INT	Không	FK → users.id
token_hash	CHAR(64)	Không	UNIQUE
expires_at	TIMESTAMP(6)	Không	
used	BOOLEAN	Không	mặc định: false
created_at	TIMESTAMP(6)	Không	mặc định: now()
used_at	TIMESTAMP(6)	Có	

B.5. customer_profiles

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
user_id	INT	Không	PK; FK → users.id
default_address_id	INT	Có	FK → addresses.id
preferred_store_id	INT	Có	FK → laundry_stores.id
preferred_service_id	INT	Có	FK → services.id
detergent_preference	ENUM(regular, fragrance_free, sensitive_skin)	Không	mặc định: regular
water_temperature_preference	ENUM(cold, warm, hot)	Không	mặc định: warm
fold_preference	ENUM(fold, hang, no_preference)	Không	mặc định: no_preference
notification_channel_preference	ENUM(email, push, in_app, sms)	Không	mặc định: in_app
loyalty_points	INT	Không	mặc định: 0
membership_tier	ENUM(bronze, silver, gold)	Không	mặc định: bronze
date_of_birth	DATE	Có	
gender	ENUM(male, female, other)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.6. addresses

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
user_id	INT	Không	FK → users.id
label	VARCHAR(50)	Không	mặc định: "Nhà riêng"
recipient_name	VARCHAR(100)	Không	
phone	VARCHAR(20)	Không	
address_line	VARCHAR(255)	Không	
ward	VARCHAR(100)	Có	
district	VARCHAR(100)	Có	
city	VARCHAR(100)	Có	
latitude	DECIMAL(10, 7)	Có	
longitude	DECIMAL(10, 7)	Có	
is_default	BOOLEAN	Không	mặc định: false
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.7. laundry_stores

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
name	VARCHAR(150)	Không	
address	VARCHAR(255)	Không	
latitude	DECIMAL(10, 7)	Có	
longitude	DECIMAL(10, 7)	Có	
description	TEXT	Có	
phone	VARCHAR(20)	Có	
image_url	VARCHAR(255)	Có	
cancellation_hours_before	INT	Không	mặc định: 2
cancellation_refund_percent	SMALLINT	Không	mặc định: 100
status	ENUM(active, inactive)	Không	mặc định: active
created_by	INT	Có	FK → users.id
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.8. staff_assignments

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
staff_id	INT	Không	FK → users.id
store_id	INT	Không	FK → laundry_stores.id
assigned_at	TIMESTAMP(6)	Không	mặc định: now()

Ràng buộc duy nhất tổ hợp: (staff_id, store_id).

B.9. store_schedules

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
store_id	INT	Không	FK → laundry_stores.id
day_of_week	SMALLINT	Không	
open_time	TIME(6)	Không	
close_time	TIME(6)	Không	
slot_duration_minutes	INT	Không	mặc định: 60
max_bookings_per_slot	INT	Không	mặc định: 5

Ràng buộc duy nhất tổ hợp: (store_id, day_of_week).

B.10. store_schedule_exceptions

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
store_id	INT	Không	FK → laundry_stores.id
exception_date	DATE	Không	
is_closed	BOOLEAN	Không	mặc định: true
open_time	TIME(6)	Có	
close_time	TIME(6)	Có	
reason	VARCHAR(255)	Có	

Ràng buộc duy nhất tổ hợp: (store_id, exception_date).

B.11. services

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
store_id	INT	Có	FK → laundry_stores.id
name	VARCHAR(100)	Không	
description	TEXT	Có	
pricing_unit	ENUM(per_kg, per_item)	Không	mặc định: per_kg
unit_price	DECIMAL(10, 2)	Không	
estimated_turnaround_hours	INT	Không	mặc định: 24
status	ENUM(active, inactive)	Không	mặc định: active
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.12. machines

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
store_id	INT	Không	FK → laundry_stores.id
name	VARCHAR(100)	Không	
type	ENUM(washer, dryer, washer_dryer)	Không	
capacity_kg	DECIMAL(6, 2)	Không	mặc định: 7.00
status	ENUM(idle, running, maintenance, offline)	Không	mặc định: idle
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.13. machine_usage_logs

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
machine_id	INT	Không	FK → machines.id
booking_id	INT	Không	FK → bookings.id
started_at	TIMESTAMP(6)	Không	
ended_at	TIMESTAMP(6)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

B.14. bookings

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
booking_code	VARCHAR(20)	Không	UNIQUE
customer_id	INT	Không	FK → users.id
store_id	INT	Không	FK → laundry_stores.id
service_id	INT	Không	FK → services.id
service_name	VARCHAR(100)	Không	
pricing_unit	ENUM(per_kg, per_item)	Không	
unit_price	DECIMAL(10, 2)	Không	
in_store	BOOLEAN	Không	mặc định: true
pickup_address_id	INT	Không	FK → addresses.id
delivery_address_id	INT	Không	FK → addresses.id
scheduled_pickup_start	TIMESTAMP(6)	Không	
scheduled_pickup_end	TIMESTAMP(6)	Không	
estimated_delivery_at	TIMESTAMP(6)	Có	
status	ENUM(pending_payment, confirmed, picked_up, processing, ready_for_delivery, delivering, completed, cancelled, no_show)	Không	mặc định: pending_payment
estimated_weight_kg	DECIMAL(6, 2)	Có	
actual_weight_kg	DECIMAL(6, 2)	Có	
estimated_price	DECIMAL(10, 2)	Không	
final_price	DECIMAL(10, 2)	Có	
discount_amount	DECIMAL(10, 2)	Không	mặc định: 0
promotion_id	INT	Có	FK → promotions.id
special_instructions	VARCHAR(255)	Có	
qr_token	VARCHAR(255)	Có	UNIQUE
cancelled_by	INT	Có	FK → users.id
cancellation_reason	VARCHAR(255)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.15. booking_items

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
booking_id	INT	Không	FK → bookings.id
item_name	VARCHAR(100)	Không	
category	VARCHAR(50)	Có	
quantity	INT	Không	mặc định: 1
unit_price	DECIMAL(10, 2)	Có	
note	VARCHAR(255)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

B.16. payments

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
booking_id	INT	Không	FK → bookings.id
payment_type	ENUM(deposit, balance, full)	Không	mặc định: full
amount	DECIMAL(10, 2)	Không	
method	ENUM(vnpay, momo, stripe, cash)	Không	
provider_transaction_id	VARCHAR(100)	Có	
status	ENUM(pending, success, failed, refunded)	Không	mặc định: pending
paid_at	TIMESTAMP(6)	Có	
refunded_at	TIMESTAMP(6)	Có	
raw_response	JSON	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.17. promotions

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
code	VARCHAR(30)	Không	UNIQUE
description	VARCHAR(255)	Có	
discount_type	ENUM(percent, fixed)	Không	
value	DECIMAL(10, 2)	Không	
valid_from	TIMESTAMP(6)	Không	
valid_to	TIMESTAMP(6)	Không	

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
usage_limit	INT	Có	
used_count	INT	Không	mặc định: 0
status	ENUM(active, expired, disabled)	Không	mặc định: active
created_by	INT	Có	FK → users.id
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.18. delivery_trips

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
booking_id	INT	Không	FK → bookings.id
trip_type	ENUM(pickup, delivery)	Không	
shipper_id	INT	Có	FK → users.id
address_id	INT	Không	FK → addresses.id
scheduled_time	TIMESTAMP(6)	Không	
started_at	TIMESTAMP(6)	Có	
completed_at	TIMESTAMP(6)	Có	
status	ENUM(unassigned, assigned, in_progress, completed, failed, cancelled)	Không	mặc định: unassigned
proof_image_url	VARCHAR(255)	Có	
note	VARCHAR(255)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

Ràng buộc duy nhất tổ hợp: (booking_id, trip_type).

B.19. reviews

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
booking_id	INT	Không	FK → bookings.id; UNIQUE
customer_id	INT	Không	FK → users.id
store_id	INT	Không	FK → laundry_stores.id
rating	SMALLINT	Không	
comment	TEXT	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

B.20. complaints

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
booking_id	INT	Không	FK → bookings.id
booking_item_id	INT	Có	FK → booking_items.id
customer_id	INT	Không	FK → users.id
type	ENUM(lost_item, damaged_item, wrong_item, late_delivery, other)	Không	
description	TEXT	Không	
evidence_image_url	VARCHAR(255)	Có	
status	ENUM(open, investigating, resolved, rejected)	Không	mặc định: open
resolution_note	VARCHAR(255)	Có	
compensation_amount	DECIMAL(10, 2)	Có	
resolved_by	INT	Có	FK → users.id
resolved_at	TIMESTAMP(6)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()
updated_at	TIMESTAMP(6)	Không	mặc định: now()

B.21. notifications

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
user_id	INT	Không	FK → users.id
type	ENUM(booking_confirmed, booking_cancelled, pickup_reminder, pickup_completed, processing_started, ready_for_delivery, delivering, delivery_completed, payment_success, payment_failed, complaint_update, waitlist_available)	Không	
channel	ENUM(email, push, in_app, sms)	Không	
title	VARCHAR(150)	Không	
message	TEXT	Không	
related_booking_id	INT	Có	FK → bookings.id
is_read	BOOLEAN	Không	mặc định: false
sent_at	TIMESTAMP(6)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

B.22. waitlist

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
customer_id	INT	Không	FK → users.id
store_id	INT	Không	FK → laundry_stores.id
desired_start_time	TIMESTAMP(6)	Không	

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
desired_end_time	TIMESTAMP(6)	Không	
status	ENUM(waiting, notified, converted, expired)	Không	mặc định: waiting
notified_at	TIMESTAMP(6)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

B.23. audit_logs

Cột	Kiểu dữ liệu	Null	Ràng buộc / ghi chú
id	INT	Không	PK; mặc định: tự tăng
actor_id	INT	Có	FK → users.id
action	VARCHAR(50)	Không	
entity_type	VARCHAR(50)	Không	
entity_id	INT	Không	
old_value	JSON	Có	
new_value	JSON	Có	
ip_address	VARCHAR(45)	Có	
created_at	TIMESTAMP(6)	Không	mặc định: now()

PHỤ LỤC C. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY

Yêu cầu: Node.js 24, pnpm, PostgreSQL và Redis đang chạy. Các bước:

```
# 1. Cài gói (tự động chạy prisma generate để sinh client)
pnpm install


# 2. Tạo tệp cấu hình từ mẫu và điền giá trị (xem Phụ lục D)
cp .env.example .env


# 3. Đồng bộ lược đồ lên cơ sở dữ liệu
pnpm prisma db push


# 4. Chạy ứng dụng (mặc định cổng 3000, tiền tố /api)
pnpm start:dev


# 5. Kiểm tra mã nguồn và chạy kiểm thử đơn vị
pnpm lint
pnpm test
```

Tạo tài khoản quản trị viên và nhân viên: hệ thống chưa có API đổi vai trò, nên sau khi đăng ký tài khoản thường, cập nhật vai trò trực tiếp trong cơ sở dữ liệu rồi đăng nhập lại để token mang vai trò mới:

```
UPDATE users SET role = 'admin'         WHERE email = 'admin@example.com';
UPDATE users SET role = 'laundry_staff' WHERE email = 'staff@example.com';
```

PHỤ LỤC D. BIẾN MÔI TRƯỜNG

Biến	Ý nghĩa	Bắt buộc
NODE_ENV	development hoặc production; production tắt log token và bật cờ Secure cho cookie	Không
PORT	Cổng lắng nghe (mặc định 3000)	Không
CORS_ORIGIN	Danh sách origin của frontend, cách nhau bởi dấu phẩy	Không
DATABASE_URL	Chuỗi kết nối PostgreSQL	Có
REDIS_HOST, REDIS_PORT, REDIS_PASSWORD, REDIS_DB	Thông tin kết nối Redis	Không
REDIS_KEY_SECRET	Khóa HMAC để băm khóa đếm giới hạn tần suất	Có
SALT_ROUNDS	Cost của bcrypt (mặc định 10)	Không
JWT_ACCESS_SECRET	Khóa ký access token	Có
JWT_ACCESS_EXPIRES_IN	Thời gian sống access token (mặc định 15m)	Không
JWT_REFRESH_SECRET	Khóa HMAC băm refresh token	Có
PASSWORD_RESET_TOKEN_SECRET	Khóa HMAC băm token đặt lại mật khẩu	Có
EMAIL_VERIFICATION_TOKEN_SECRET	Khóa HMAC băm token xác minh email	Có

Các khóa bí mật nên là chuỗi ngẫu nhiên đủ dài và khác nhau giữa các biến; tệp `.env` không được đưa lên Git.

PHỤ LỤC E. VÍ DỤ YÊU CẦU API

Đăng ký tài khoản — POST /api/auth/register

```
{ "email": "khach@example.com", "password": "Passw0rdOK", "name": "Nguyễn Văn A" }
```

Tạo cửa hàng (admin) — POST /api/stores

```
{ "name": "Giặt là Hà Đông", "address": "Nguyễn Trác, Yên Nghĩa, Hà Đông, Hà Nội",
  "phone": "0901234567", "cancellation_hours_before": 2, "cancellation_refund_percent": 100 }
```

Đặt lịch mở cửa tuần — PUT /api/stores/1/schedules

```
{ "schedules": [
  { "day_of_week": 1, "open_time": "08:00", "close_time": "18:00",
    "slot_duration_minutes": 60, "max_bookings_per_slot": 5 },
  { "day_of_week": 2, "open_time": "08:00", "close_time": "18:00" } ] }
```

Tạo dịch vụ — POST /api/stores/1/services

```
{ "name": "Giặt sấy thường", "pricing_unit": "per_kg", "unit_price": 25000,
  "estimated_turnaround_hours": 24 }
```

Xem khung giờ còn trống — GET /api/stores/1/available-slots?date=2026-10-26

```
{ "data": { "date": "2026-10-26", "is_open": true, "open_time": "08:00", "close_time": "18:00",
  "slot_duration_minutes": 60,
  "slots": [ { "start": "08:00", "end": "09:00", "capacity": 5, "booked": 0,
               "available": 5, "is_available": true }, "…" ] } }
```